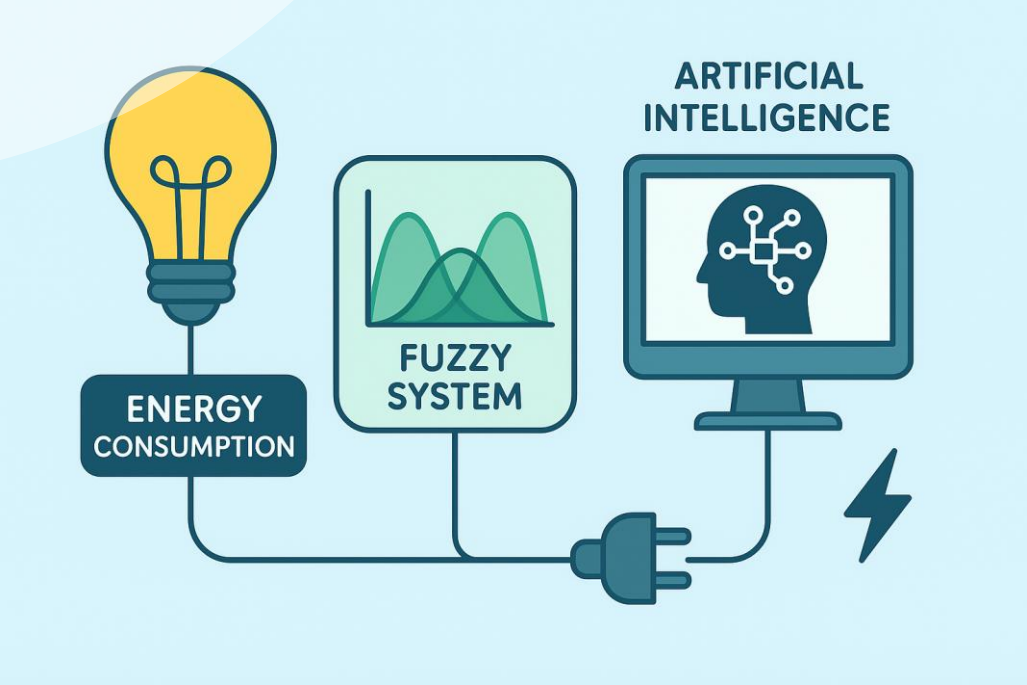


FINAL PROJECT

Artificial Intelligence Course

TITLE:
Final Project

Aylar Rahimzadeh
Fatemeh Sajed
هوش مصنوعی مقدماتی

فهرست

1. لینک های موجود
2. دیتاست اول
3. دیتاست دوم
4. نتیجه گیری نهایی

لینک های مربوطه

دیتاست اول)

https://colab.research.google.com/drive/1cGVBsZKkWjdL7RRP4K1SNnPd3Km9fcxu?usp=sharing

https://colab.research.google.com/drive/1IOs0J6BU6VNMB5hjODlPcQZMsuKVMzDx?usp=sharing

دیتاست دوم)

https://colab.research.google.com/drive/1mTuzz22jnAmeqOcqghdA09fIYELCVLXU?usp=sharing

مقدمه:

در این پروژه سعی میکنیم مصرف انرژی را ابتدا با استفاده از دیتاستی توسط الگوریتم های ماشین لرنینگ و شبکه عصبی پیشبینی کنیم سپس با استفاده از مدلسازی فازی در دیتاستی دیگر بگوییم چطور این دو میتوانند به بهبود پیش بینی یک سیستم کمک کنند.

بخش اول:

MLمدلسازی با شبکه های عصبی و

با استفاده از دیتاست اول

دیتاست اول

در این پروژه قصد تحلیل و بررسی مصرف برق در ۶ ماه اول سال ۲۰۰۷ است و تلاش برای بهینه سازی مصرف و همچنین پیش بینی مصرف در ۶ ماهه اول سال ۲۰۰۸

پیش پردازش

در مرحله اول از پردازش داده‌ها، هدف ما اصلاح فرمت تاریخ‌ها در دیتاست است. داده‌های تاریخ ممکن است شامل مشکلاتی نظیر یک رقم بودن روز یا ماه‌ها و یا دو رقمی بودن سال‌ها باشند که باید اصلاح شوند تا از اشتباهات در مدل‌سازی جلوگیری شود. برای این إصلاحات مراحل زیر طی خواهند شد:

مرحله ۱: اصلاح روزها و ماه‌های تک رقمی

ابتدا تاریخ‌ها را بررسی می‌کنیم و اگر روز یا ماه کمتر از دو رقم باشند، آنها را به صورت دو رقمی اصلاح می‌کنیم. این کار به کمک متد zfill انجام می‌شود که عدد را با صفر پر می‌کند تا طول آن به دو رقم برسد.

```
sample_dates_before = data['Date'].head(20)

print("Sample Dates before fixing:")

print(sample_dates_before)

data['Date'] = data['Date'].apply(lambda x: '/'.join([part.zfill(2) if i <
2 else part for i, part in enumerate(x.split('/'))]))
```

مرحله 2: اصلاح سال‌های دو رقمی تبدیل ۰۷ به ۲۰۰۷

در این مرحله، سال‌هایی که به صورت دو رقمی نوشته شده‌اند (مثل "07" به جای "2007") را اصلاح می‌کنیم. برای این کار، اگر سال دارای دو رقم باشد، به آن پیشوند "20" افزوده می‌شود تا سال به صورت کامل (مثلاً 2007) نمایش داده شود.

```
data['Date'] = data['Date'].apply(lambda x: x[:6] + '20' + x[6:] if
len(x.split('/')[2]) == 2 else x)
```

در مرحله بعد داده ها را بررســـی کرده و به داده‌های عددی تبدیل می‌کنیم و داده هایی که به عدد تبدیل نمی‌شوند را حذف می‌کنیم زیرا تعداد قابل توجهی نداشتند.

```
data['Global_active_power'] = pd.to_numeric(data['Global_active_power'],
errors='coerce')

data['Global_reactive_power'] =
pd.to_numeric(data['Global_reactive_power'], errors='coerce')

data['Voltage'] = pd.to_numeric(data['Voltage'], errors='coerce')

data['Global_intensity'] = pd.to_numeric(data['Global_intensity'],
errors='coerce')

data['Sub_metering_1'] = pd.to_numeric(data['Sub_metering_1'],
errors='coerce')

data['Sub_metering_2'] = pd.to_numeric(data['Sub_metering_2'],
errors='coerce')

data['Sub_metering_3'] = pd.to_numeric(data['Sub_metering_3'],
errors='coerce')

data.drop(['index'],axis=1,inplace=True)

data.info()
```

در این بخش از کد، هدف ترکیب دو ســـتون تاریخ (Date) و زمان (Time) به یک ســـتون واحد به نام Datetimeاست تا داده‌ها به صورت یکپارچه و استاندارد در قالب تاریخ و زمان ذخیره شوند

```
data['Datetime'] = pd.to_datetime(data['Date'] + ' ' + data['Time'],
format='%d/%m/%Y %H:%M:%S', dayfirst=True)

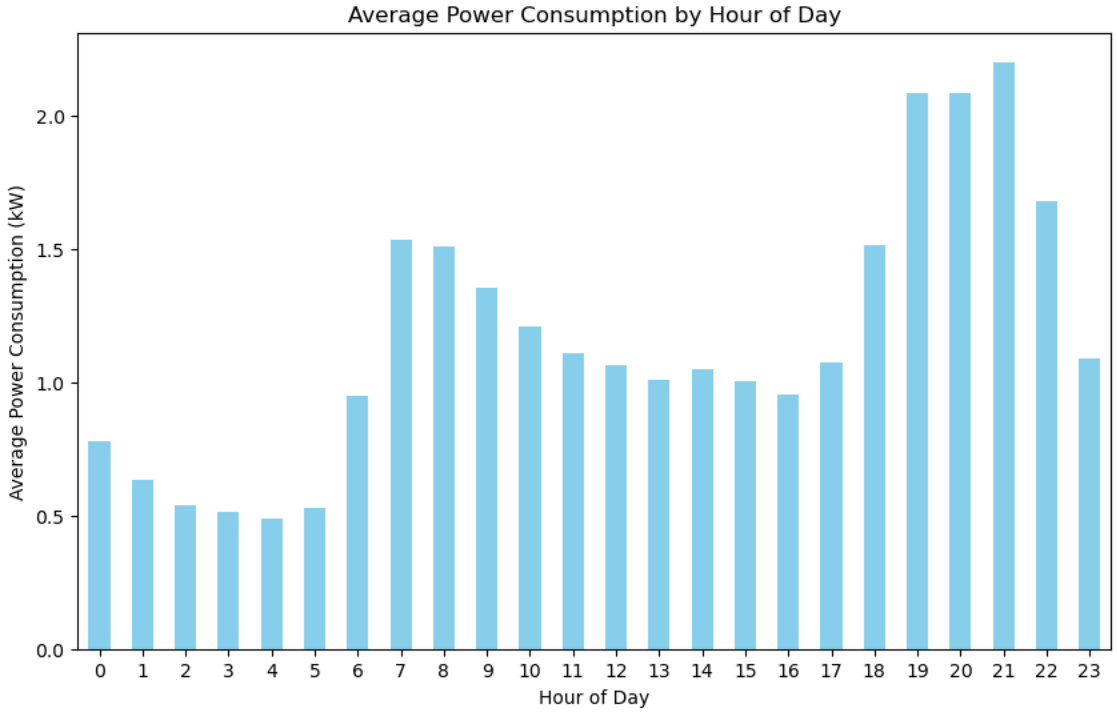
data.drop(columns=['Date', 'Time'], inplace=True)
```

بررسی آماری

در قسمت دوم این تحلیل داده‌ها را از لحاظ آماری بررسی می‌کنیم، در ابتدا بررسی میکنیم در چه ساعتی از روز مصرف بیشتر بوده سپس همین تحلیل را برای روزهای هفته انجام می‌دهیم.

```
data['Hour'] = data['Datetime'].dt.hour
```

```
hourly_consumption = data.groupby('Hour')['Global_active_power'].mean()
plt.figure(figsize=(10, 6))
hourly_consumption.plot(kind='bar', color='skyblue')
plt.title('Average Power Consumption by Hour of Day')
plt.xlabel('Hour of Day')
plt.ylabel('Average Power Consumption (kW)')
plt.xticks(rotation=0)
plt.show()
```



مشاهده می‌شود در ساعت ۲۱ بیتشرین مصرف اتفاق افتاده و در کمترین مصرف در ساعت ۴ صبح بوده است.

در ادامه بررسی میکنیم در این ساعت مصرف در کدام بخش بیشتر بوده است.

برای روزهای هفته:

```
data['DayOfWeek'] = data['Datetime'].dt.day_name()


daily_consumption =
data.groupby('DayOfWeek')['Global_active_power'].mean()


ordered_days = ['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday',
'Saturday', 'Sunday']

daily_consumption = daily_consumption[ordered_days]


plt.figure(figsize=(10, 6))

daily_consumption.plot(kind='bar', color='lightcoral')

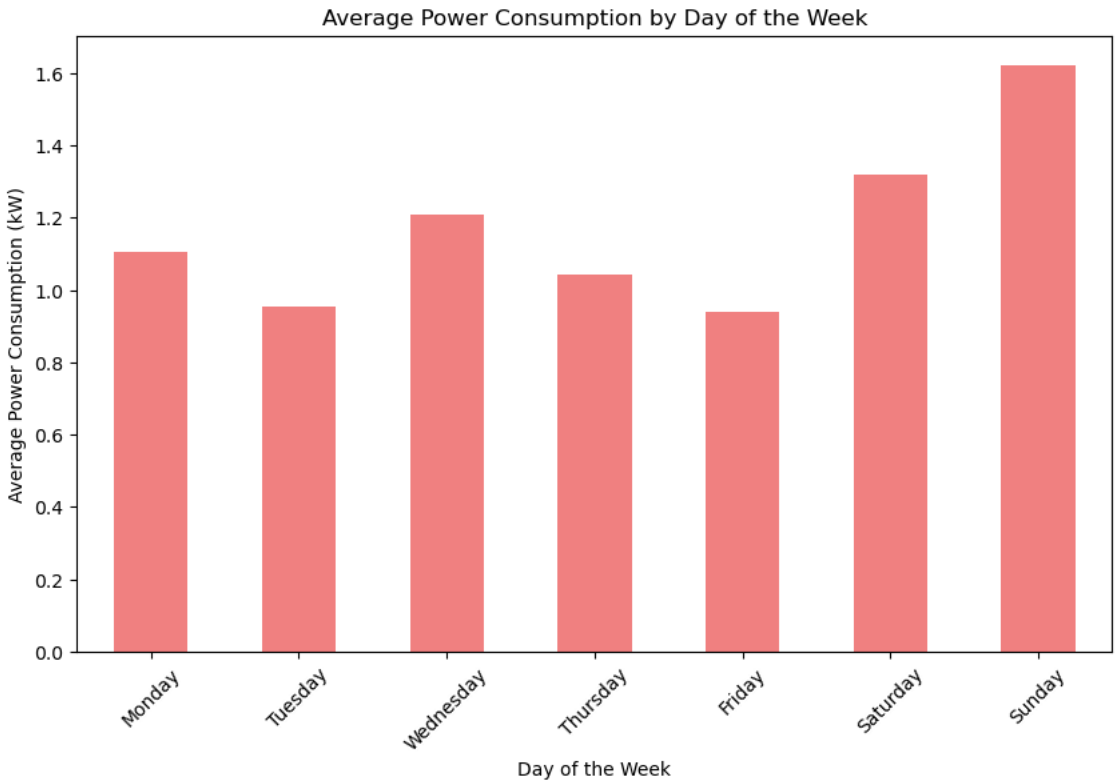
plt.title('Average Power Consumption by Day of the Week')

plt.xlabel('Day of the Week')

plt.ylabel('Average Power Consumption (kW)')

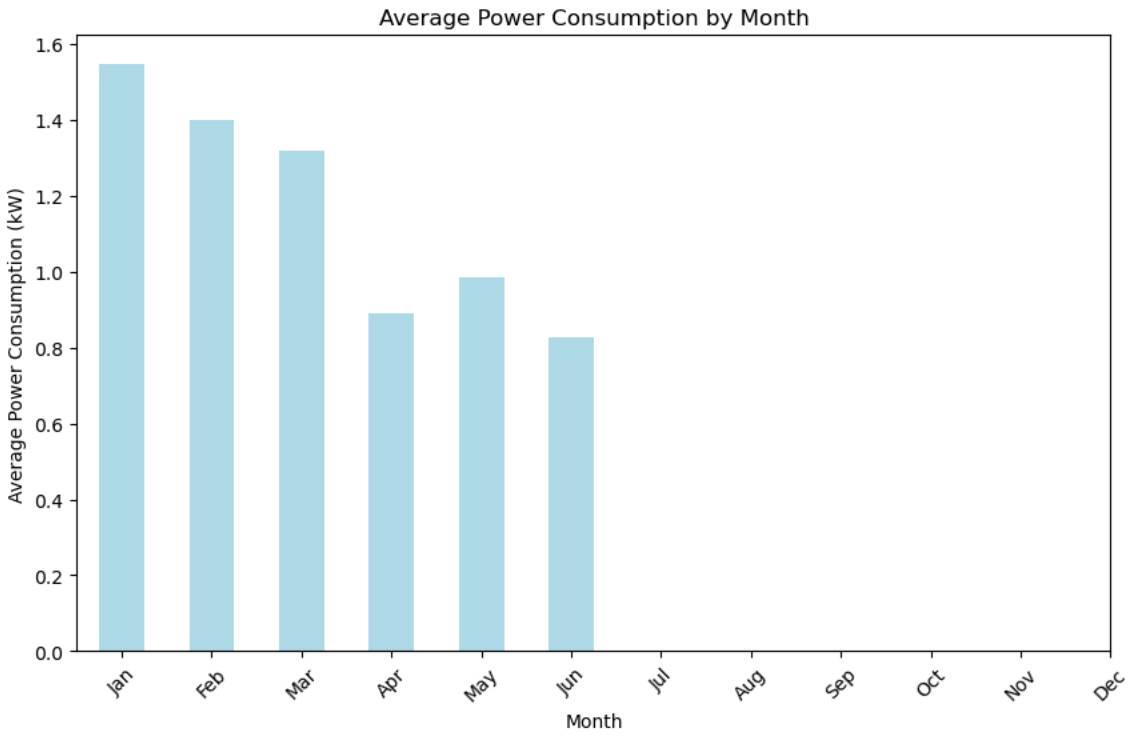
plt.xticks(rotation=45)

plt.show()
```



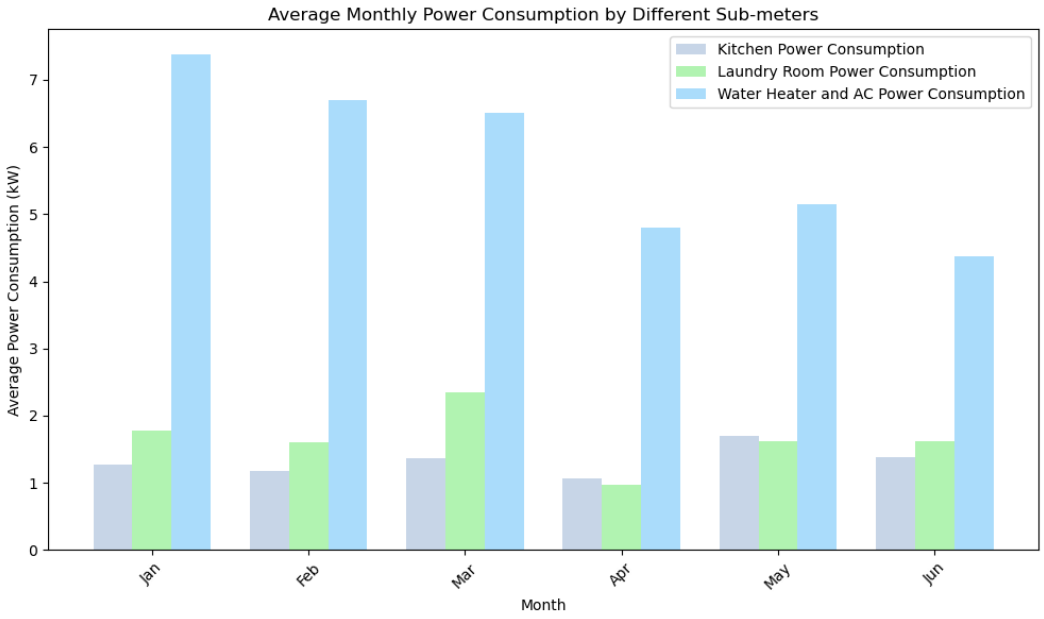
مصرف برق در روز تعطیل یعنی یکشنبه از باقی روزها بیشتر بوده است.

در ادامه نمودار به تفکیک هر ماه آورده شده است.



مشاهده می‌شود در ماه اول سال که ماه سر هست مصرف انرژی بالاتر بوده است و به تدریج انرژی مصرفی کاهش یافته است، در ادامه به تفکیک مشخص می‌کنیم اختلاف در مصرف کدام بخش باعث ایجاد این تفاوت شده‌است.

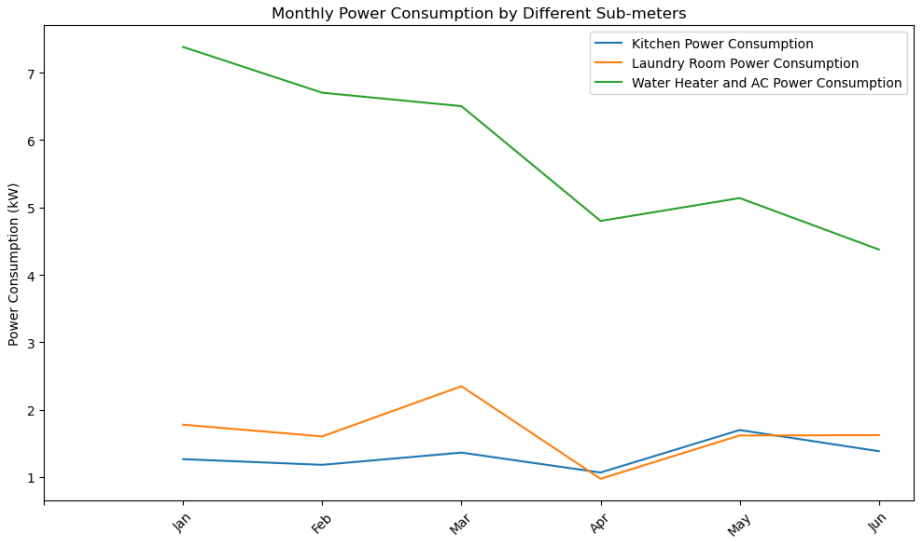
در این قسمت میزان مصرف هر بخش رو به صورت جداگانه نمایش می‌دهیم.

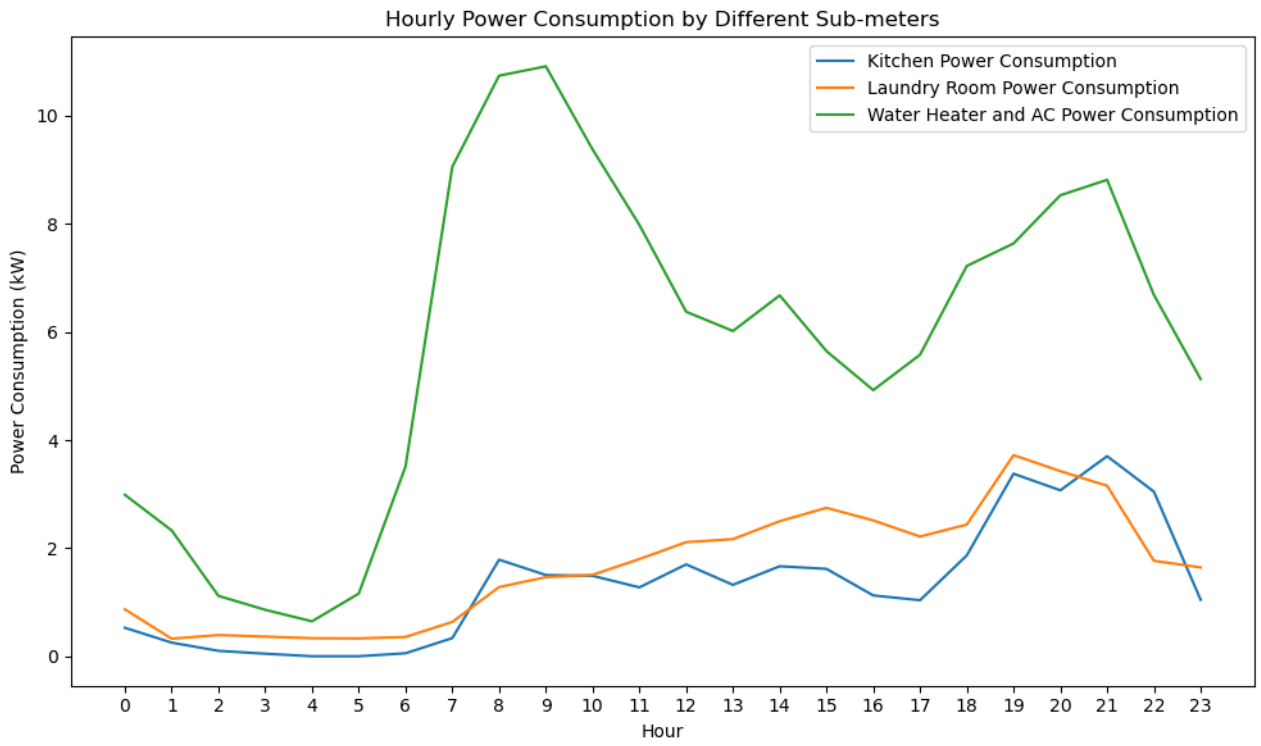


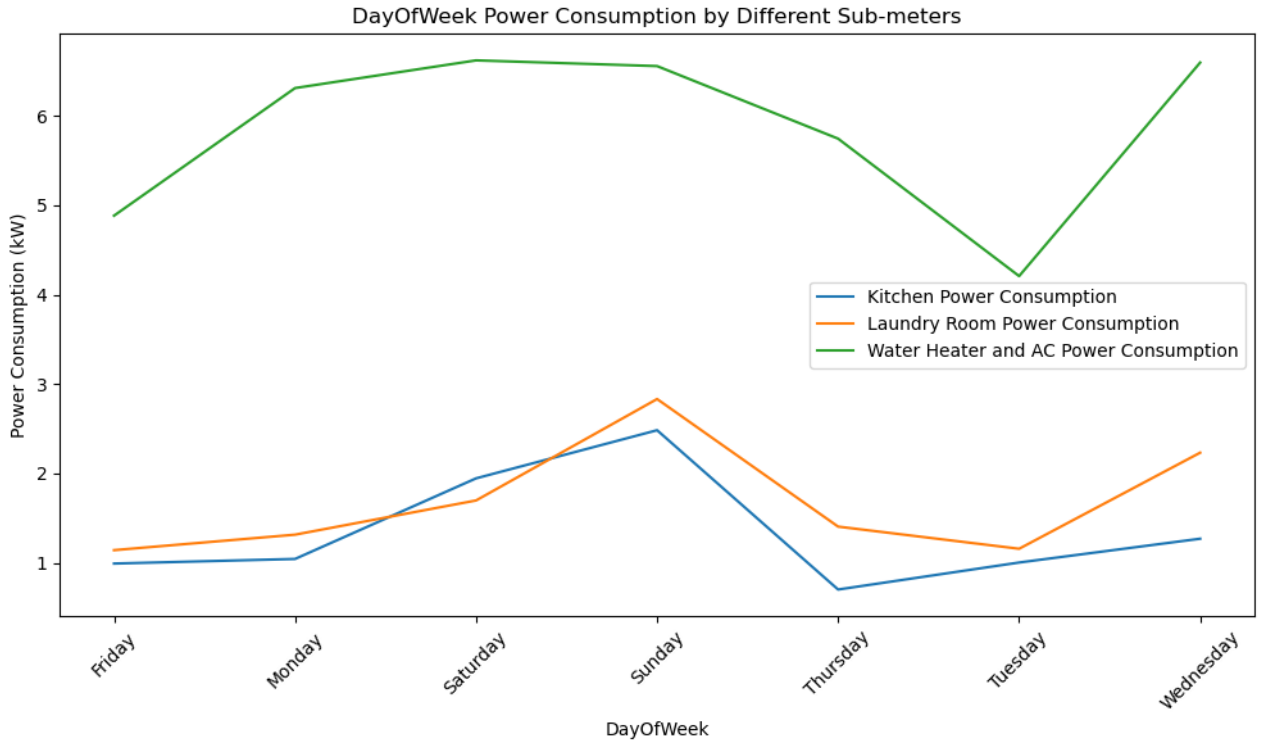
همانطور که مشاهده می‌شود میزان مصرف در بخش آشپزخانه و اتاق شست و شو کمتر از قسمت اب گرم کن بوده است.

همچنین رنج مصرف در آشپزخانه تقریبا در هر ماه یکسان است اما مصرف ابگرمکن با گذر زمان و گرم شدن هوا کاهش یافته است و تفاوت عمده در ماه اول به دلیل مصرف انرژی ابکرمکن بوده است.

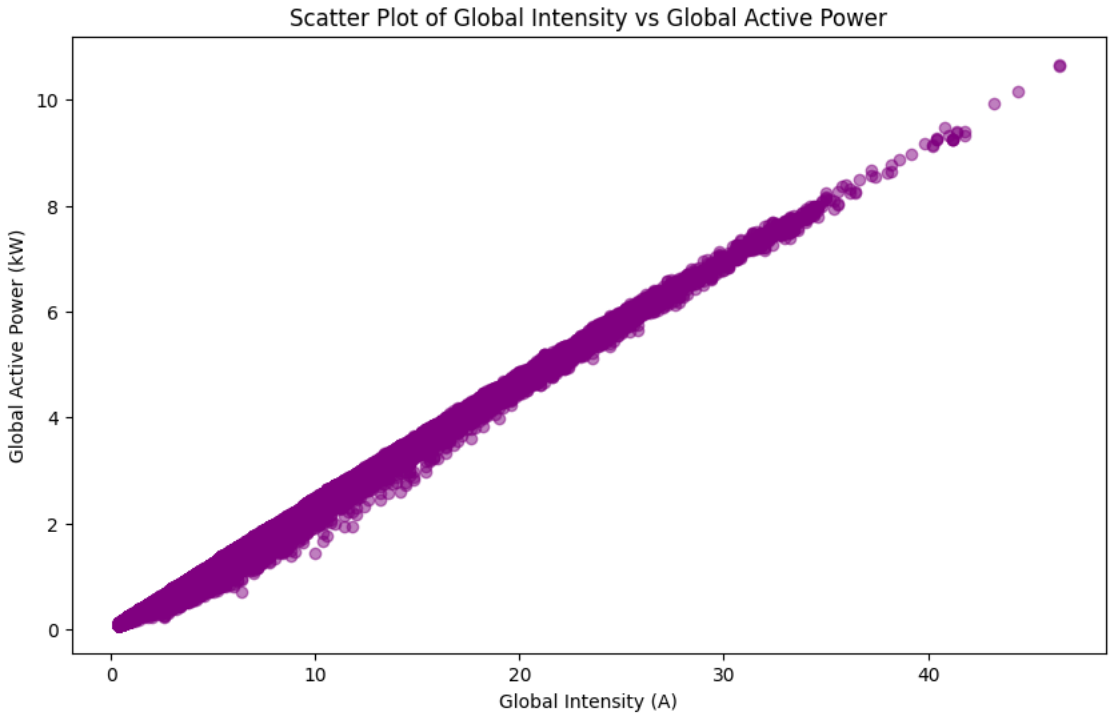
در ادامه روند مصرف هر یک از ایتم ها را به صورت روزانه و ساعتی نیز بررسی میکنیم.



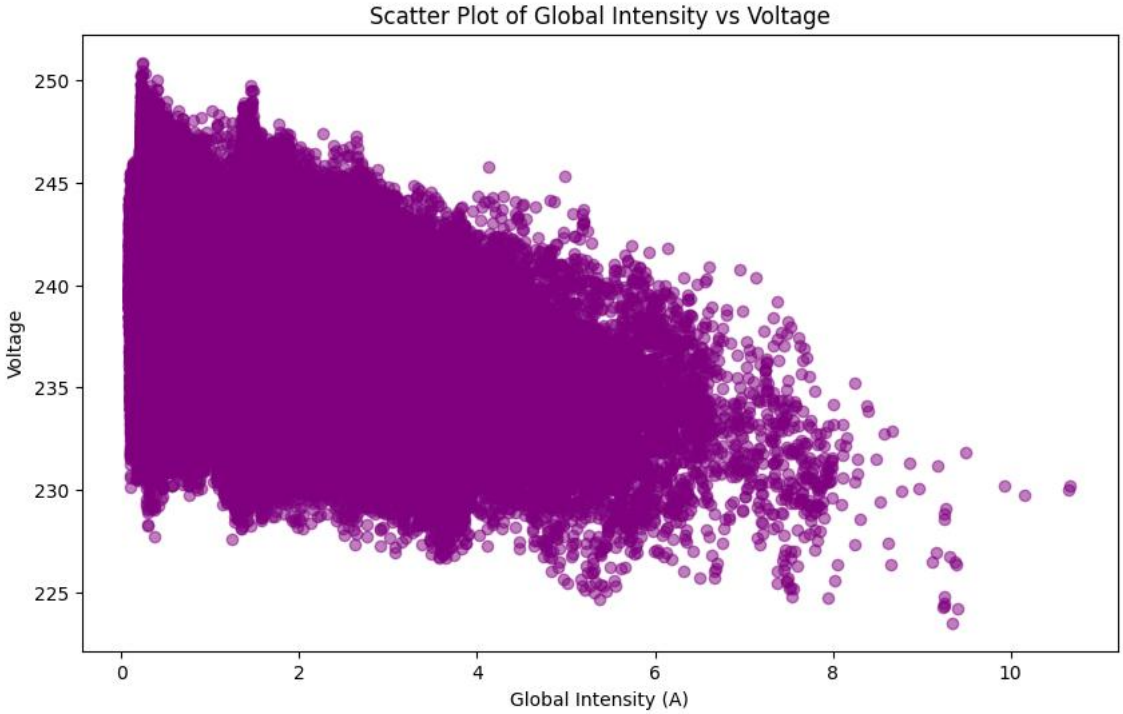




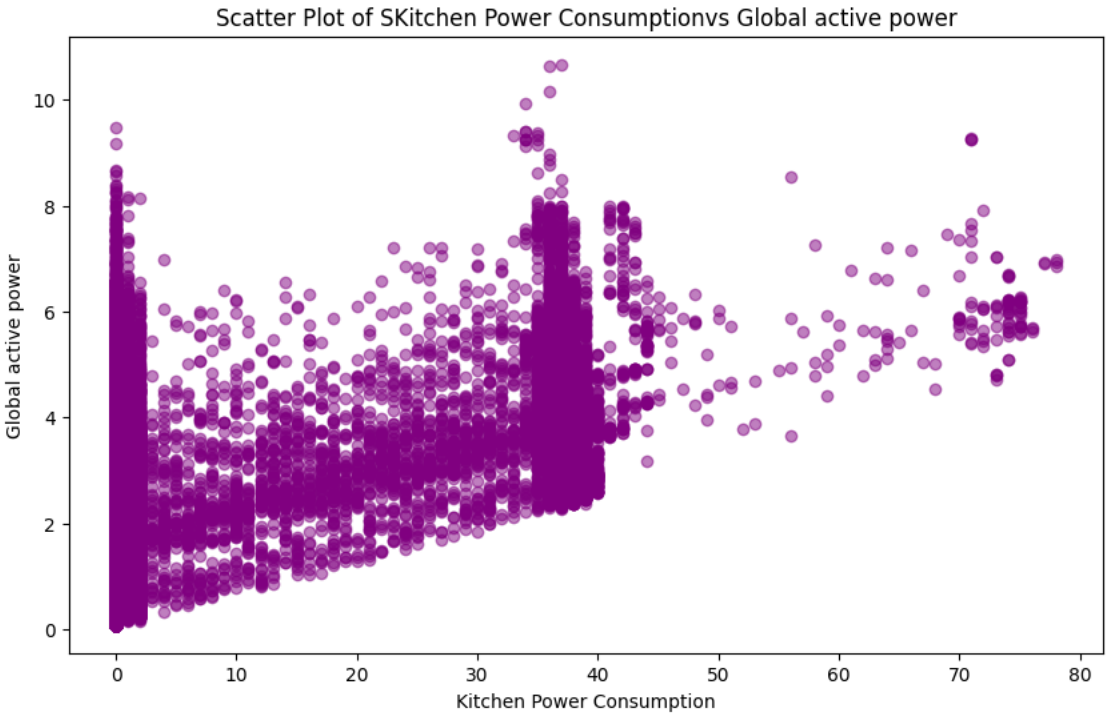
می‌توان روند تغییرات هر یک از این پارمترها را براساس انرژی مصرفی کلی را بررسی کرد.



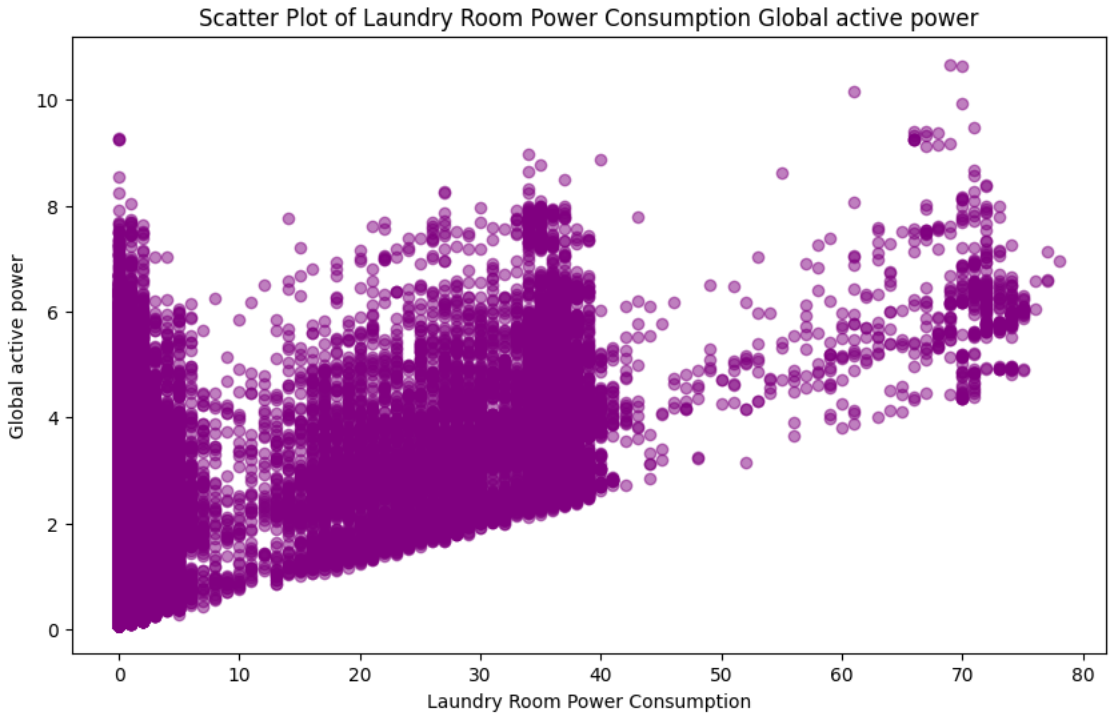
همانطور که مشاهده می‌شود با افزایش انرژی مصرف شده جریان نیز افزایش پیدا کرده است.



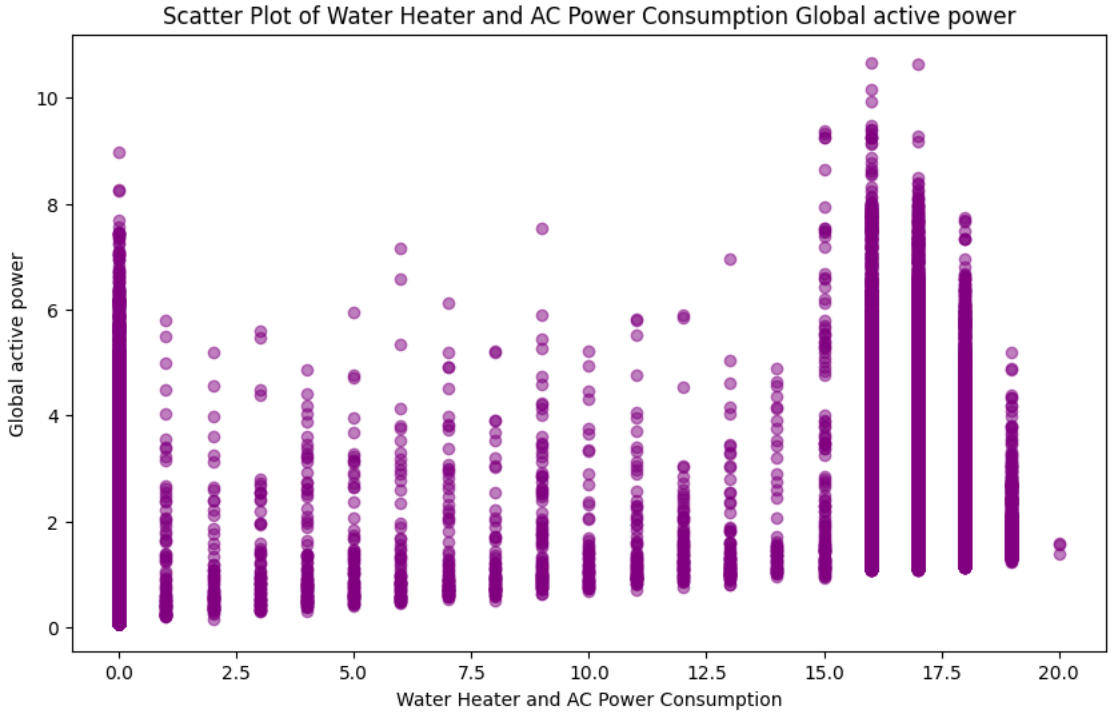
در این مور دیده می‌شود با افزایش جریان بار با افت ولتاژ مواجه هستیم و این نمودار نشان دهنده یم ابر متراکم در مرکز است. این یعنی در بیشتر مواقع مصرف در یک محدوده مشخص اتفاق می‌افتد، ولی گاهی افت ولتاژ شدیدتر یا شدت جریان بسیار بالا رخ می‌دهد.



به طور کلی، با افزایش مصـرف توان آشـپزخانه، توان فعال جهانی افزایش می‌یابد. در محدوده‌های خاصـی، نقاط به‌طور فشرده متمرکز شده‌اند، مخصوصاً در مقادیر بین ۰ تا ۴۰ که نشان‌دهنده‌ی مصرف‌های کم و متوسط است که بیشـتر در عمل رخ می‌دهند. در اعداد بالاتر از ۴۰، نقاط پراکندگی بیشـتری وجود دارند و انرژی مصـرفی کلی می‌تواند تا حدود ۱۰ کیلووات هم افزایش یابد.

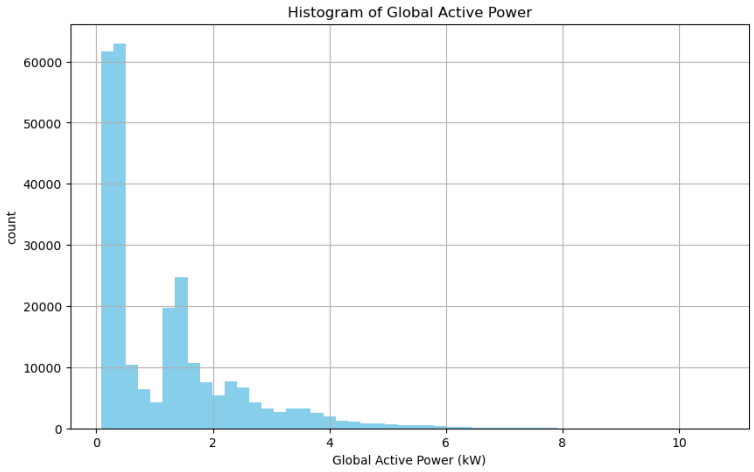


تحلیل این نمودار نیز مانند نمودار مصرف انرژی در آشپزخانه است.



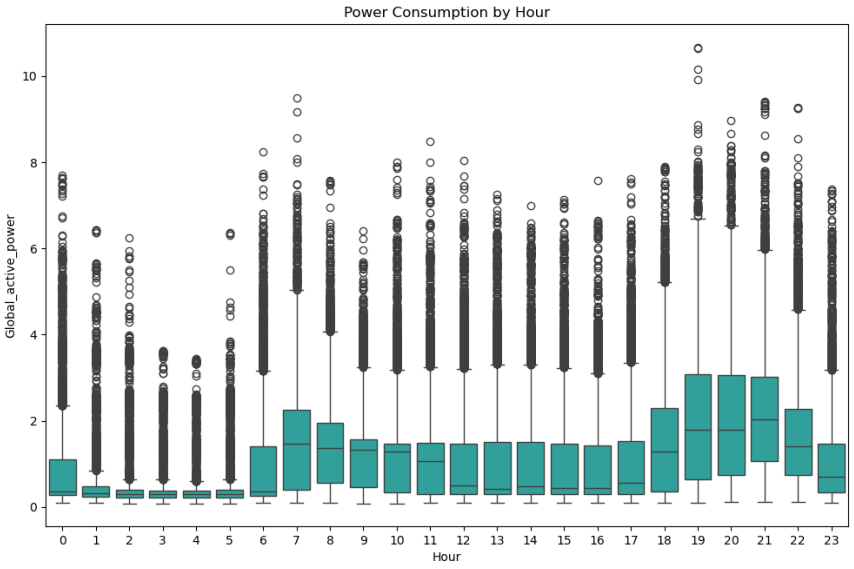
در نزدیکی صفر محور افقی، نقاط بسیار فشرده و متراکم هستند که به معنای این است که در این حالت‌ها، میزان مصرف دستگاه‌ها تقریباً صفر است، ولی توان مصرفی فعال جهانی همچنان بالاست. این نشان می‌دهد که ممکن است حتی زمانی که مصرف دستگاه‌ها پایین است، سیستم همچنان انرژی مصرف می‌کند، این اتفاق می‌تواند به دلیل مصرف بقیه دستگاه ها باشد یا وضعیت اماده به کار همین بخش.

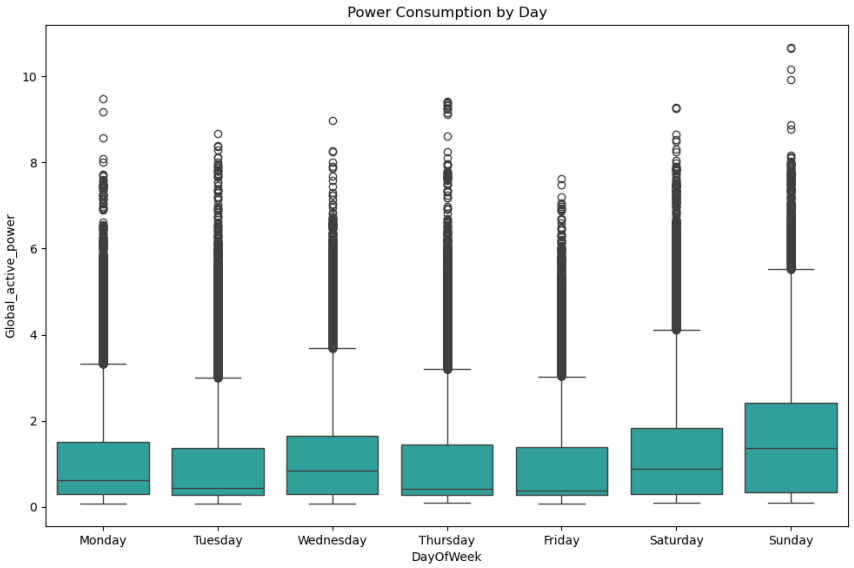
در این قسمت می‌بینیم اگر داده‌های مصرف بین ۰ تا ۲ کیلو وات قرار گرفته اند که در رسم نمودار جعبه ای نیز به همین برداشت می‌رسیم.

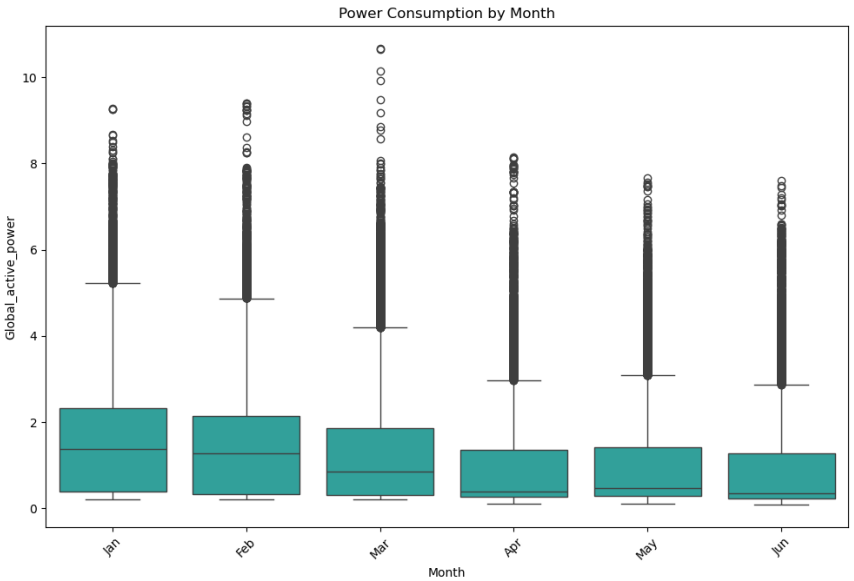


نمودار جعبه‌ای داده‌ها به صورت زیر است:

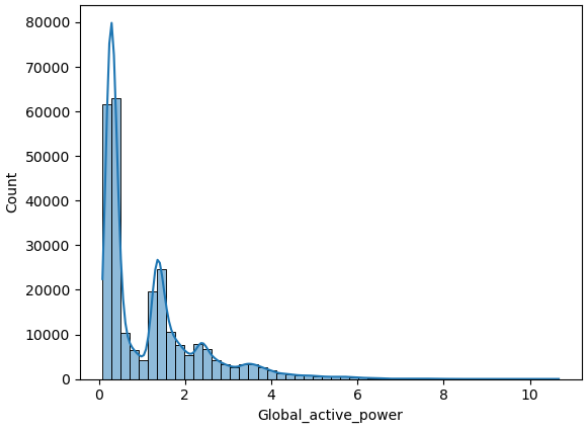
تعداد داده های خارج از این رنج زیاد بوده و نمی‌توان آنها را به عنوان داده های پرت در نظر گرفت، پس ادامه تحلیل را به همراه این داده ها ادامه می‌دهیم

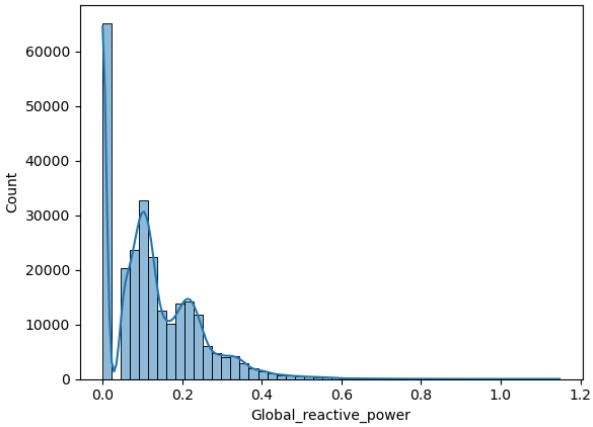

Power Consumption by Hour
Global_active_power
Hour
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23
0 2 4 6 8 10

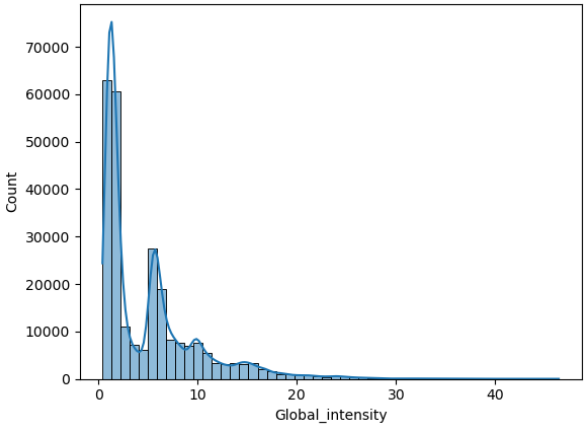

Power Consumption by Day
Global_active_power
DayOfWeek
Monday
Tuesday
Wednesday
Thursday
Friday
Saturday
Sunday
0 2 4 6 8 10

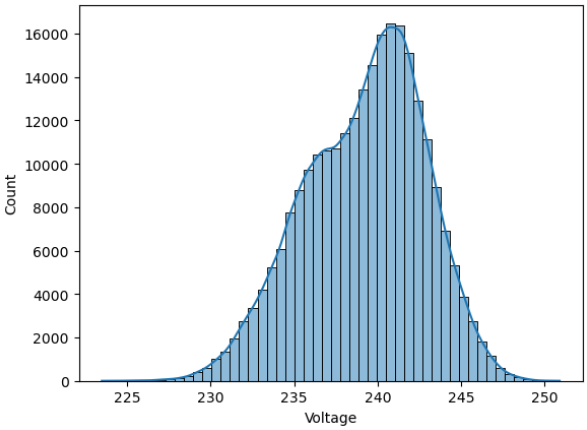

Power Consumption by Month
Global_active_power
Month
Jan
Feb
Mar
Apr
May
Jun
0 2 4 6 8 10

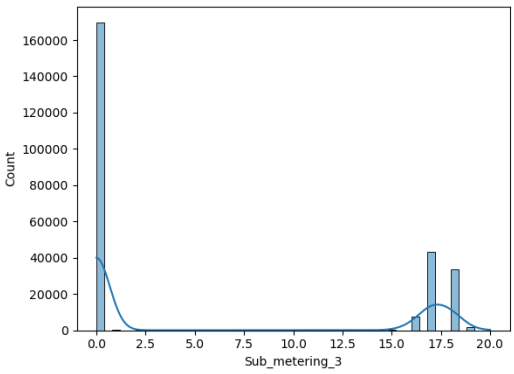
در خصوص نرمال بودن داده‌ها نمودار توزیع آنها را رسم کرده و با توجه به انها این امر رو بررسی میکنیم

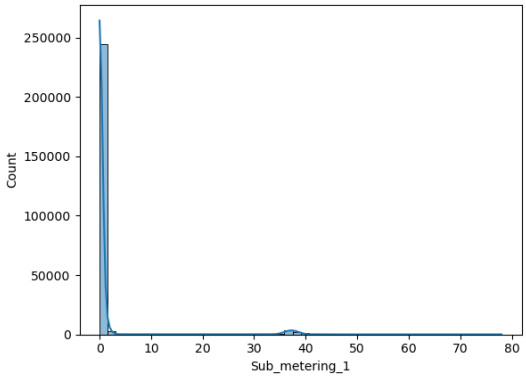


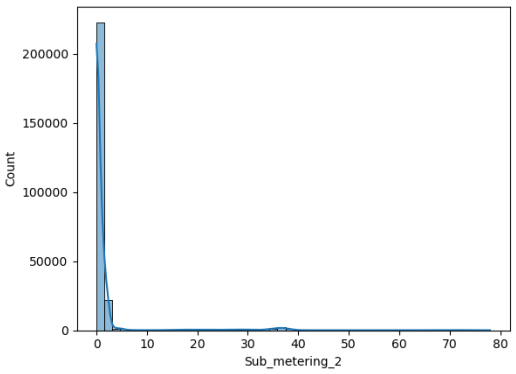












چند متد مختلف برای نرمال کردن و استاندارد کردن داده ها استفاده می‌کنیم اما داده‌ها به علت پوشش داده‌ها منحرف بودن از توزیع نرمال صرفا داده‌ها را در یک بازه مشخص قرارمیدهیم تا مدلمان به درستی کار کند.

لگاریتمی:

```
model_var =['Global_active_power', 'Global_reactive_power', 'Voltage',
'Global_intensity', 'Sub_metering_1', 'Sub_metering_2', 'Sub_metering_3']

for x in model_var:

    sns.histplot(

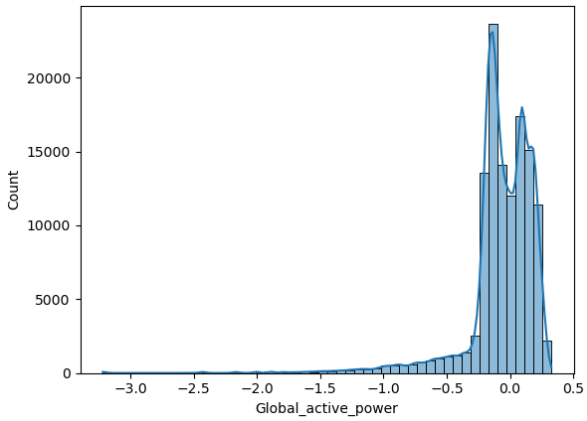
        data=np.log10(data[x]),

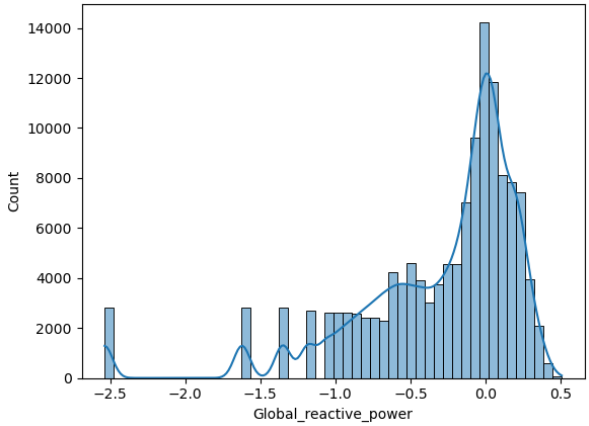
        bins=50,

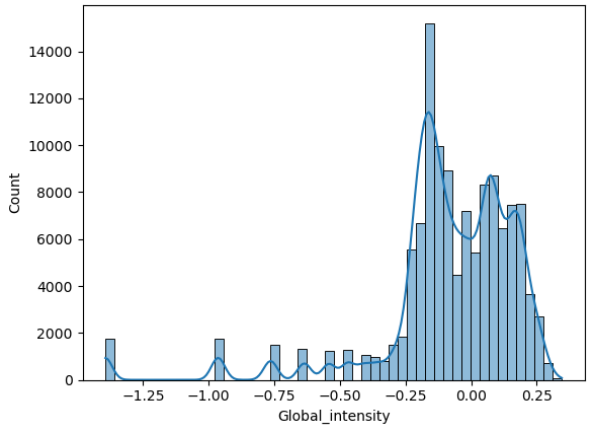
        kde=True

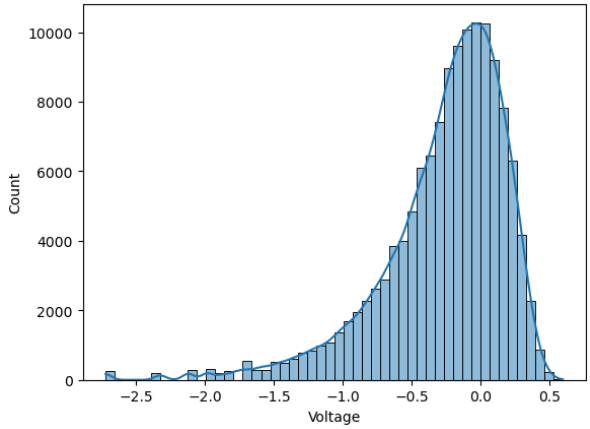
    )

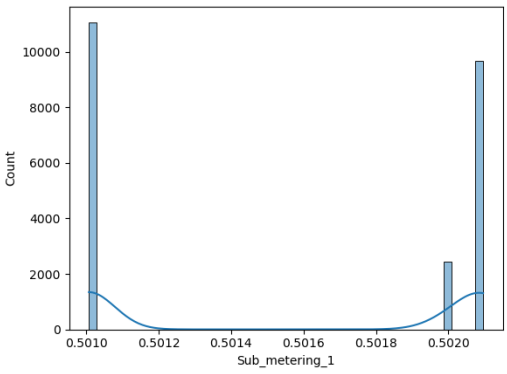
    plt.show()
```

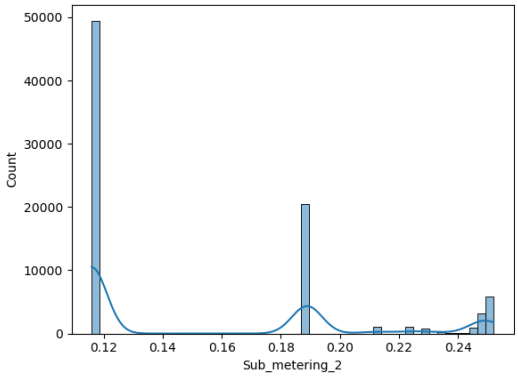


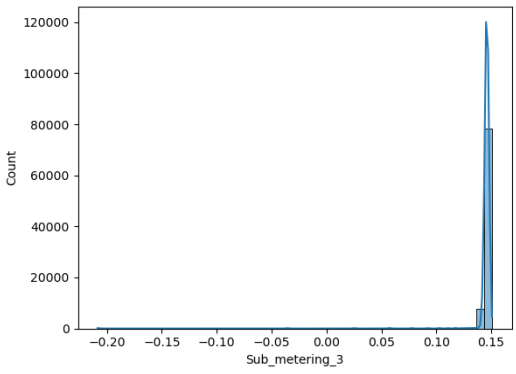












Minmax scaler

```
scaler = MinMaxScaler()

for x in model_var:

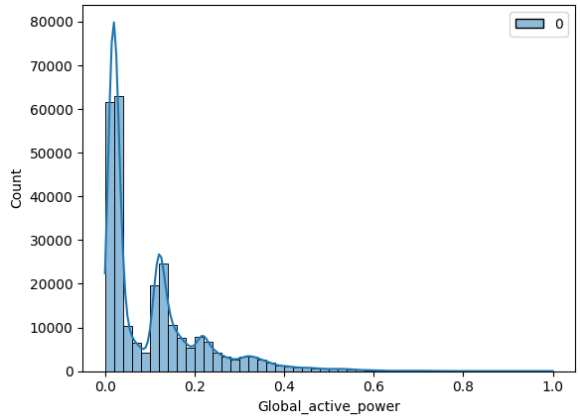
    # Reshape the data to 2D for scaling
```

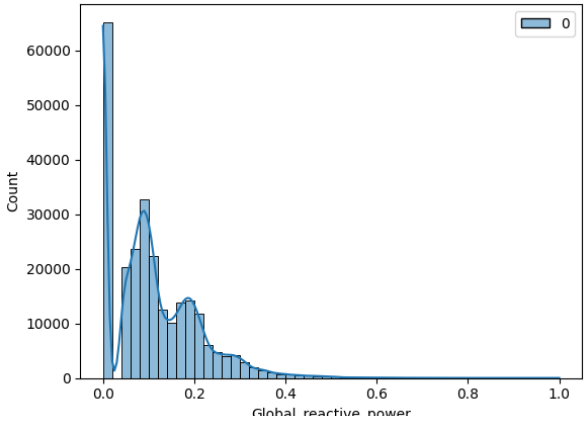
```
    data_scaled = scaler.fit_transform(data[[x]])  # Use double brackets
to select a single column as DataFrame

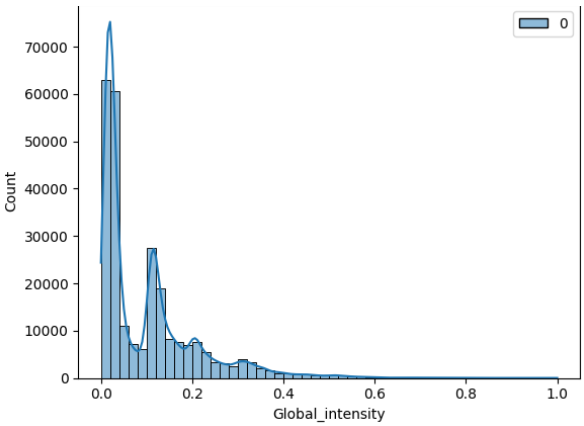
    sns.histplot(data_scaled, bins=50, kde=True)

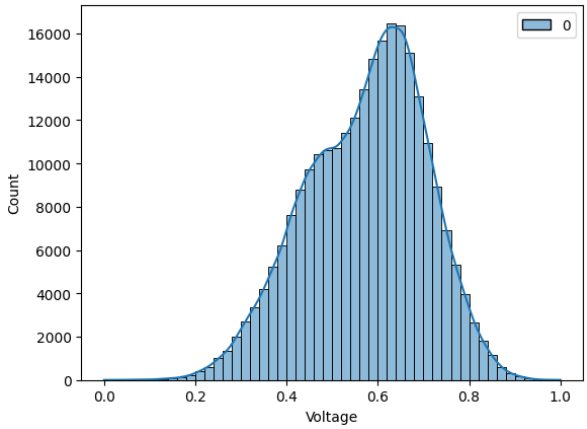
    plt.xlabel(x)

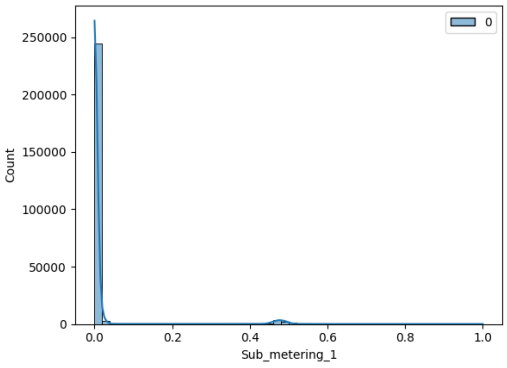
    plt.show()
```

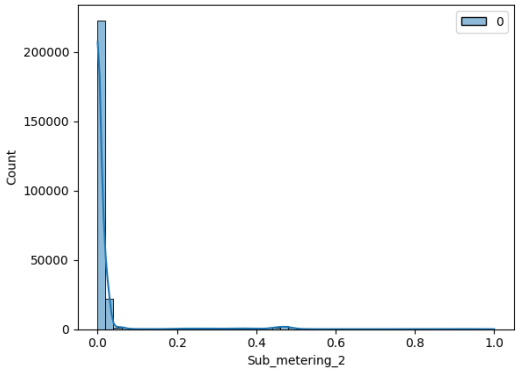


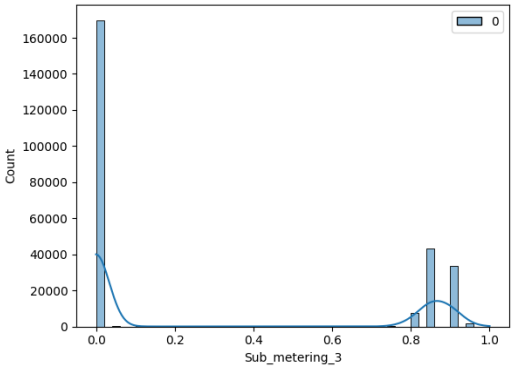












همچنین جندین روش دیگر برای استاندارد سازی استفاده شده است که در ادامه کدهای آنها آورده شده است.

Inverse Transformation

```
for x in model_var:
    sns.histplot(
        data=1/data[x],
        bins=50,
        kde=True
    )
    plt.show()
```

Square Root Transformation

```
for x in model_var:
    sns.histplot(
        data=np.sqrt(data[x]),
        bins=50,
        kde=True
    )
    plt.show()
```

آموزش و آزمایش

در ادامه با اســتفاده از فیچرهای بدســت آمده چند مدل را آموزش میدهیم با بتوانیم از بین آنها یکی را انتخاب کرده و به وسیله آن حجم مصرف در آینده را تخمین بزنیم .

برای این کار از میانگین حجم مصرف روزانه ماهانه و ساعتی استفاده میکنیم.

به جهت اینکه هر روز برتری ای نسبت به باقی روزها نداشته باشد ساعت‌ها روزهای هفته و ماه ها را کد گذاری کرده در این حالت به داده های آموزش و تست تقسیم بندی می‌کنیم.

```
X_encoded = X.drop(columns=['DayOfWeek'])
```

```
X_encoded = pd.concat([X_encoded, encoded_df], axis=1)
X_encoded = X_encoded.drop(columns=['Hour'])
X_encoded = pd.concat([X_encoded, encoded_df], axis=1)
X_encoded = X_encoded.drop(columns=['Month'])
X_encoded = pd.concat([X_encoded, encoded_df], axis=1)
```

در پیش بینی مدل ها با این ویژگی ها دقت خوبی را بدست نیاوردیم نتایج به طور خلاصه در جدول زیر امده است.

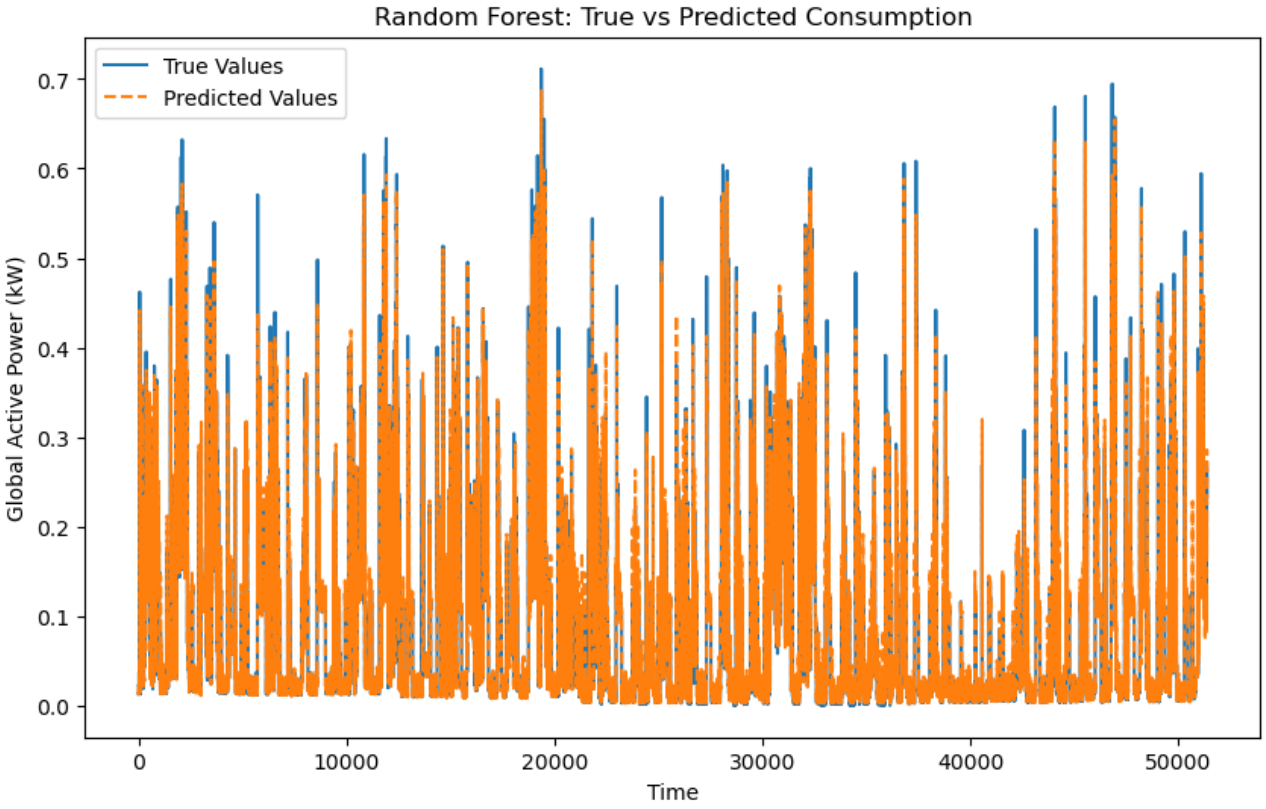
	Linear Regression	Decision Tree	Random Forest	MLP
MSE	1.339016176840927	1.339016176840928	1.33900613116445	1.3390512850868312
R^2	0.03177132569579699	0.031771325695796215	0.03177858962000202	0.03174593928824132

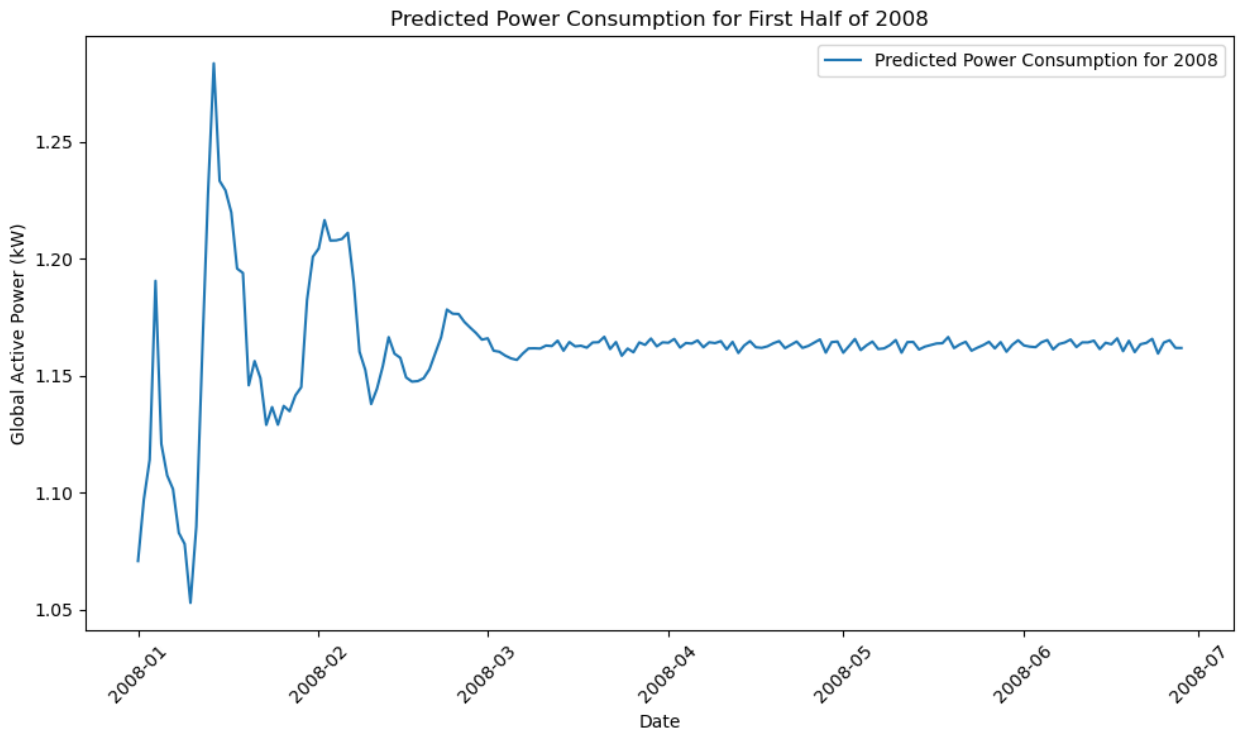
همانطور که مشاهده می‌شود نتایج بدست آمده همگی بسیار به یکدیگر نزدیک اند و همگی غیرقابل قبول هستند پس در ادامه روش را تغییر میدهیم.

در ادامه از ویژگی‌های ساعتی استفاده می‌کنیم و این ویژگی رو منحصرا برای هر ساعت و روز جداگانه در نظر میگیریم و داده های هر ۲۴ ساعت را یک نقطه در نظر می‌گیریم. در ابتدا با الگوریتم رندوم فارست پیش بینی را انجام می‌دهیم با توجه به مقیاس‌های عددی بدست آمده در این قسمت این الگوریتم عملکرد خوبی از خود نشان داده است پس با همین الگوریتم ۶ ماه اول سال ۲۰۰۸ را تخمین میزنیم از آنجایی که در این حالت تعداد داده‌ها خیلی زیاد است و آموزش برای هر مدل حدود نیم ساعت طول میکشید پیش بینی را با همین مدل انجام دادیم.

```
Mean Absolute Error (MAE): 0.010143583015141388
Mean Squared Error (MSE): 0.0007217481982046524
Root Mean Squared Error (RMSE): 0.02686537173025254
```

در این سه تست مشاهده می‌کنیم که خطا بشدت پایین است این می‌تواند نشانه اورفیت شدن باشد اما در اینجا به دلیل اینکه هر ساعت رو ما جداگانه در نظر گرفتیم همچین احتمالی به وجود نمی‌آید.


Random Forest: True vs Predicted Consumption
True Values
Predicted Values
Global Active Power (kW)
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0
10000
20000
30000
40000
50000
Time



در این تصـویر که تخمین ۶ ماه اول سـال ۲۰۰۸ اسـت می‌توان گفت تا ماه سـوم پیش بینی خیلی خوب انجام شده و روندی نزدیک به روند سال قبل دارد اما از ماه سوم به بعد داده ها به نوعی یکنواخت شده اند و تغییرات آنها در بازه محدودی قرار گرفته است، پس میتوان گفت از ماه سوم به بعد مدل به درستی پیش بینی نمی‌کند.

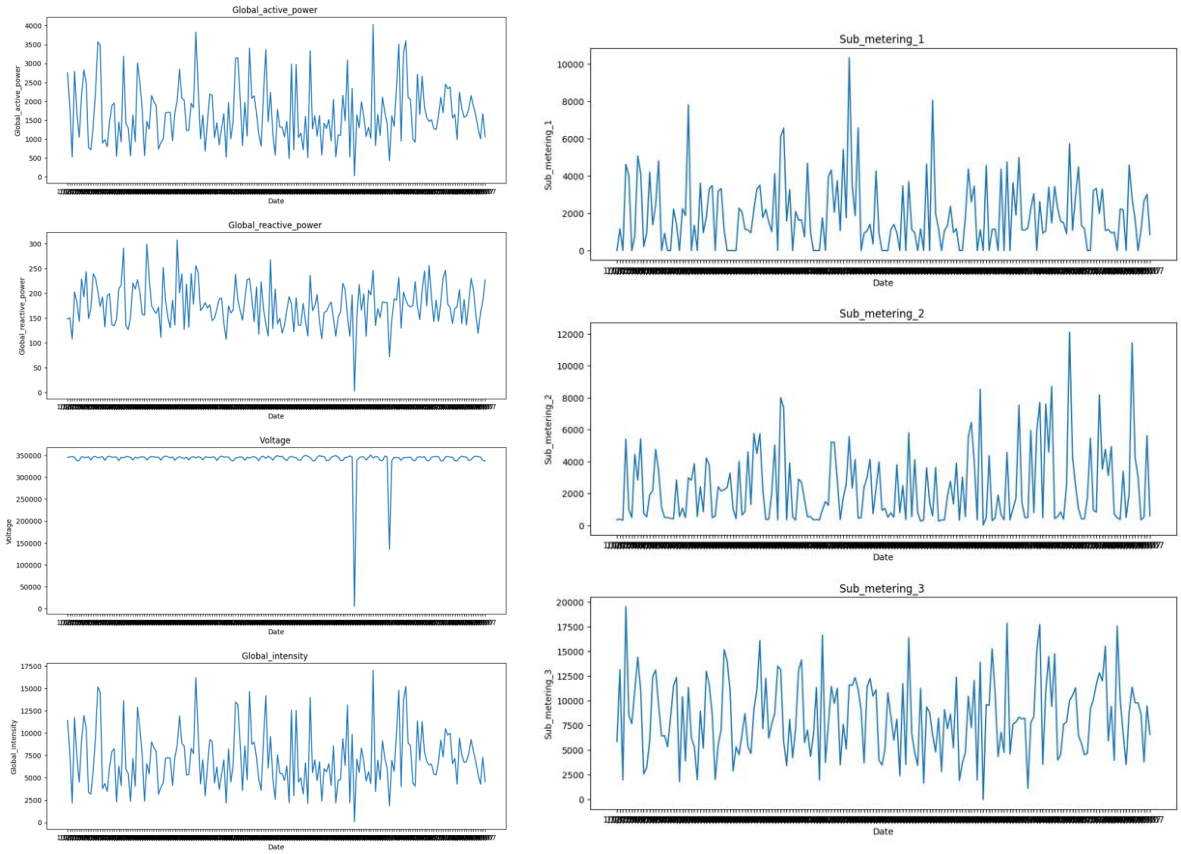
در این مسائل در ابتدا قصد ما پیش بینی ۶ ماه دوم سال بود اما این دیتا ست به دلیل محدودیت‌هایی که داشت مانند نداشـتن دمای هوا این امکان رو برای ما فراهم نمیکرد و نتایج بدسـت امده برای ۶ ماه دوم کاملا غیر قابل قبول بودند زیرا حتی اگر مدل به خوبی کار میکرد در نهایت قادر به پیش بینی ۶ ماه دوم بر اسـاس ۶ ماه اول بود و ۶ ماه دوم نیز مانند ۶ ماه اول محاسبه می‌شد.

سری زمانی

در ادامه یک فایل حاوی کدهای پیش بینی بر اسـاس سـری زمانی قرار دارد که توضـیحات آن نیز به شـرح زیر است.

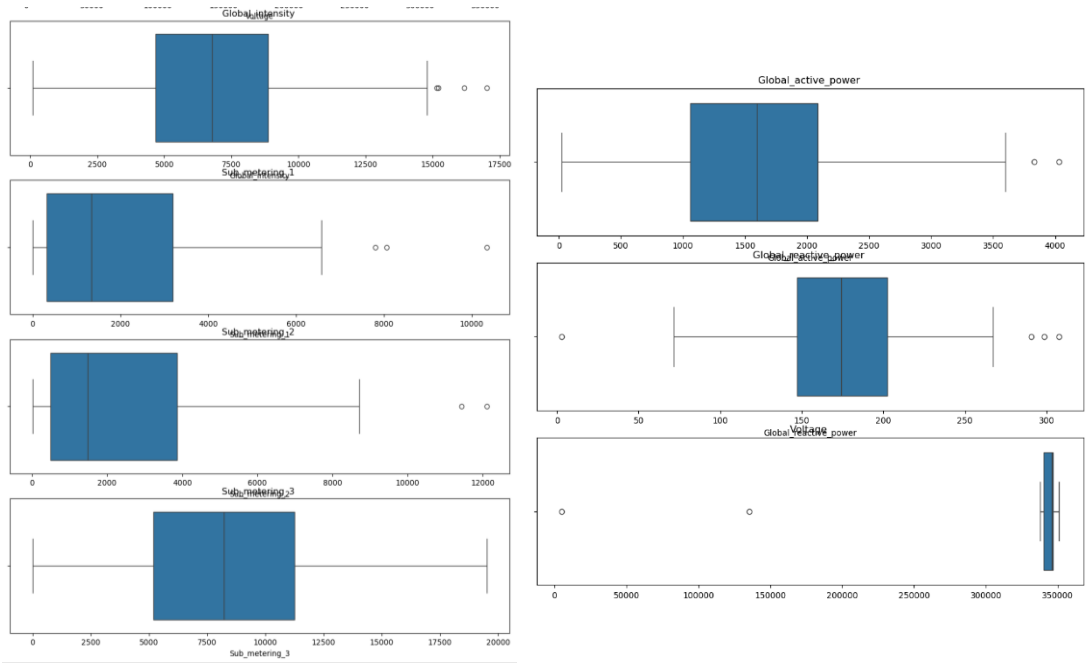
در این الگوریتم برای سنگین نشدن پردازش‌ها داده ها را بر اساس تاریخ دسته بندی می‌کنیم در آن صورت داده های ما از هر ساعت نخواهند بود و روزانه هستند.

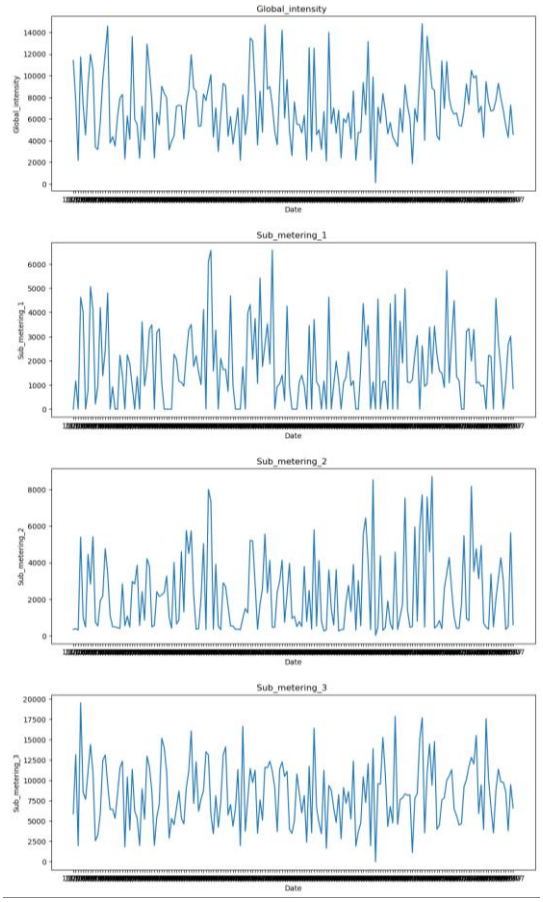
سپس داده های سری زمانی را رسم می‌کنیم.

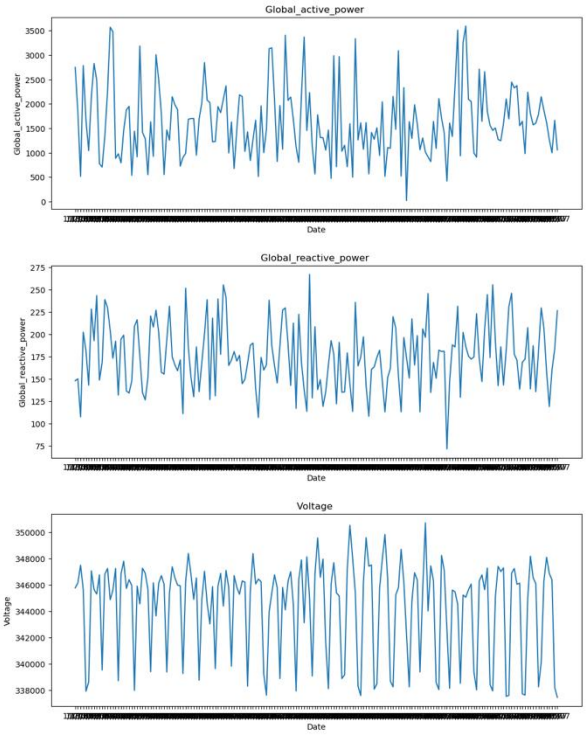


ســـپس با توجه به نمودار جعبه ای روزهایی که داده هایی خارج از رنج دارند را حذف کرده و دوباره بدون این داده ها نمودارها را رسم می‌کنیم.

مشاهده می‌شود که به حذف شدن مقادیر پرت بخصوص در ولتاژ نمودار بهتر می‌شود.




Global_intensity
Global_intensity
Date
Sub_metering_1
Sub_metering_1
Date
Sub_metering_2
Sub_metering_2
Date
Sub_metering_3
Sub_metering_3
Date


Global_active_power
Global_active_power
Date
Global_reactive_power
Global_reactive_power
Date
Voltage
Voltage
Date

بخش دوم:

مدلسازی فازی با استفاده از دیتاست دوم

درک و پردازش اولیه داده‌ها

- **مقایسه با پیش‌بینی مصرف برق :**ابتدا باید دیتاست مربوط به پیش‌بینی مصرف برق خانگی را بررسی کرده و میبینیم چه ویژگی‌هایی با این دیتاست مشترک هستند. ویژگی‌هایی مثل دما، رطوبت، و شاخص راحتی کاربر ممکن است رابطه‌ای با مصرف برق خانگی داشته باشند.
- **پیش‌پردازش داده‌ها :**بررسی داده‌ها از لحاظ وجود مقادیر گم‌شده یا داده‌های پرت و اصلاح آنها. این شامل نرمال‌سازی یا مقیاس‌دهی ویژگی‌ها نیز می‌شود.

تحلیل فازی

- **تعریف متغیرهای فازی :**برای تحلیل فازی، ابتدا باید متغیرهای ورودی را به دسته‌های فازی تقسیم می شود.:
  - **دما :**به سه دسته "کم"، "متوسط" و "زیاد" تقسیم میشود.
  - **رطوبت :**می‌تواند به سه دسته "پایین"، "متوسط" و "بالا" تقسیم شود.
  - **مصرف انرژی یا راحتی کاربر :**مشابه همین فرآیند.
- **استفاده از قوانین فازی :**برای هر ورودی، قوانینی مثل "اگر دما بالا باشد و رطوبت کم باشد، مصرف انرژی بیشتر می‌شود" را تعریف کنید.

ترکیب دو دیتاست

- **ادغام ویژگی‌ها :**پس از پردازش و تحلیل فازی، می‌توانید ویژگی‌های جدیدی از دیتاست ایجاد کنید که شامل شاخص‌های فازی محاسبه شده باشد. این ویژگی‌ها می‌توانند به مدل پیش‌بینی مصرف برق خانگی شما اضافه شوند.
- **یادگیری ماشین یا مدل‌سازی پیش‌بینی :**می‌توانید از الگوریتم‌های یادگیری ماشین مثل رگرسیون، درخت تصمیم یا شبکه‌های عصبی برای پیش‌بینی مصرف برق خانگی با استفاده از داده‌های فازی و داده‌های پیش‌بینی‌شده استفاده میشود.

پیش پردازش داده های فازی:

بررسی خود دیتا:

	Timestamp	Temperature_C	Humidity_%	CO2_ppm	Occupancy_Count	External_Temperature_C	Kp	Ki	Kd	Fuzzy_Adjustment_Factor	ISA_Optimiz
0	2017-01-01 00:00:00	22.494481	44.945633	983.998649	12	7.752784	2.055758	0.113566	0.448615	0.850555	
1	2017-01-01 00:10:00	29.408572	43.316484	547.609596	26	14.710904	0.995900	0.205418	0.164713	1.056811	
2	2017-01-01 00:20:00	26.783927	37.046157	677.311755	37	31.403525	0.863097	0.916411	0.315834	0.991201	
3	2017-01-01 00:30:00	25.183902	54.290667	930.624509	32	26.358835	1.077068	0.888923	0.287849	0.809292	
4	2017-01-01 00:40:00	19.872224	49.064966	785.671476	5	28.574897	2.340311	0.849772	0.492452	1.196332	

بررسی ستون های دیتاست:

پیش پردازش دیتاست:

```
missing_values = data.isnull().sum()
print("Missing values in the dataset:")
print(missing_values)
```

در این قسمت مشاهده می کنیم که آیا داده از دست رفته ای داریم یا نه تا اگر داریم برای جایگزینی آن تصمیمی بگیریم. در خروجی مشاهده شد که دیتاست هیچ داده NULL ندارد.

```
Missing values in the dataset:
Timestamp                     0
Temperature_C                 0
Humidity_%                    0
CO2_ppm                       0
Occupancy_Count               0
External_Temperature_C        0
Kp                            0
Ki                            0
Kd                            0
Fuzzy_Adjustment_Factor       0
ISA_Optimization_Score        0
HVAC_Power_Consumption_kWh    0
Cooling_Heating_Output_C      0
Response_Time_s               0
Energy_Efficiency_%           0
User_Comfort_Index            0
HVAC_Efficiency_Class         0
dtype: int64
```

بررسی رنج داده‌ها و انتخاب روش نرمال‌سازی

برای انتخاب روش نرمال‌سازی مناسب، باید ابتدا رنج مقادیر موجود در هر ستون را بررسی کنیم. اگر داده‌ها به صورت گسسته و با مقادیر محدود باشند، از روش‌هایی مانند **Min-Max Scaling** یا **Standardization** استفاده می‌شود.

مرحله بعدی :

- تبدیل ویژگی‌های عددی به فازی
- استفاده از ترکیب ویژگی‌ها برای تولید ویژگی‌های جدید
- حذف ویژگی‌های غیرضروری

در این قسمت از کد ما بازه داده ها را مشخص کردیم تا نحوه نرمال سازی بررسی شود. بازه ها به این صورت بودند:

```
Range of data (Min, Max):
     Temperature_C  Humidity_%      CO2_ppm  Occupancy_Count  \
min      18.000140   30.006310   400.038499              0.0
max      29.996612   69.996993  1199.920782             49.0

     External_Temperature_C        Kp        Ki        Kd  \
min                5.002798  0.500258  0.100043  0.010056
max               34.993913  2.499723  0.999906  0.499948

     Fuzzy_Adjustment_Factor  ISA_Optimization_Score  \
min                 0.800112               50.016555
max                 1.199946               99.995599

     HVAC_Power_Consumption_kWh  Cooling_Heating_Output_C  Response_Time_s  \
min                    2.001831                 -1.998841         5.006078
max                    9.999336                  1.998244        29.995145

     Energy_Efficiency_%  User_Comfort_Index  HVAC_Efficiency_Class
min            80.000041                 6.0                    0.0
max            97.998844                 9.0                    2.0
```

در مرحله بعد تابعی مینوسیم تا در مورد نرمال سازی این داده ها تصمیم گیری کند:

```
scaling_method = {}
for column in data.columns:
    if data[column].dtype in ['int64', 'float64']:
        min_value = data[column].min()
        max_value = data[column].max()


        if max_value - min_value > 10:
            scaling_method[column] = 'Min-Max Scaling'
        else:
            scaling_method[column] = 'Standardization'


print("\nSuggested scaling method for each feature:")
print(scaling_method)
```

در این قسمت از کد با توجه به پراکندگی ها متد موردنظر شناسایی میشود طبق نتایج مشخص شد که:

برای ویژگی‌هایی که تفاوت زیاد بین min و max دارند مثل دما، رطوبت، CO2_ppmو HVAC_Power_Consumption_kWh، **Min-Max Scaling** مناسب‌تر است.

برای ویژگی‌هایی که دامنه محدودی دارن مثل "User_Comfort_Index"، **Standardization** مناسب‌تر است.

اعمال نرمال سازی:

در این قسمت از کد با توجه به متد انتخاب شده روی هر دیتا نرمال سازی صورت میگیرد.

ابتدا از دیتاست کپی میگیریم تا دیتای اولیه دست نخورده باقی بماند سپس با توجه به حلقه ای که داریم برای هر ستون و متد انتخابی نظیر آن یک به یک جلو میرویم و نرمال سازی با متد انتخاب شده برای آن را روی آن پیاده سازی میکنیم با استفاده از fit و سپس نمایش می دهیم.

```
scaler_dict = {
    'Min-Max Scaling': MinMaxScaler(),
    'Standardization': StandardScaler()
}

scaled_data = data.copy()

for column, method in scaling_method.items():
    if method == 'Min-Max Scaling':
        scaler = scaler_dict[method]
        scaled_data[column] = scaler.fit_transform(data[[column]])
    elif method == 'Standardization':
        scaler = scaler_dict[method]
        scaled_data[column] = scaler.fit_transform(data[[column]])

# نمایش داده‌های نرمال‌سازی شده
print("\nScaled data preview:")
print(scaled_data.head())
```

خروجی نهایی:

```
Scaled data preview:
             Timestamp  Temperature_C  Humidity_%   CO2_ppm  Occupancy_Count  \
0  2017-01-01 00:00:00       0.374639    0.373570  0.730058         0.244898
1  2017-01-01 00:10:00       0.950982    0.332832  0.184491         0.530612
2  2017-01-01 00:20:00       0.732198    0.176037  0.346643         0.755102
3  2017-01-01 00:30:00       0.598823    0.607250  0.663330         0.653061
4  2017-01-01 00:40:00       0.156053    0.476577  0.482112         0.102041

   External_Temperature_C        Kp        Ki        Kd  \
0                0.091693  0.954611 -1.688958  1.371281
1                0.323699 -0.888653 -1.337428 -0.625012
2                0.880285 -1.119617  1.383654  0.437612
3                0.712079 -0.747488  1.278454  0.240834
4                0.785969  1.449495  1.128618  1.679523

   Fuzzy_Adjustment_Factor  ISA_Optimization_Score  \
0                -1.303457                0.604025
1                 0.490879                0.663068
2                -0.079904                0.052042
3                -1.662421                0.328049
4                 1.704645                0.103999

   HVAC_Power_Consumption_kWh  Cooling_Heating_Output_C  Response_Time_s  \
0                    1.269636                  0.483984         0.087446
1                   -1.277062                  1.301895         0.749777
2                    0.708067                 -0.242240         0.549169
3                   -0.754289                  1.358335         0.033143
4                    0.319982                  0.471683         0.242976

   Energy_Efficiency_%  User_Comfort_Index  HVAC_Efficiency_Class
0             0.002228            0.443435               0.532975
1             0.990634           -1.340295               0.532975
2             0.760978           -1.340295               0.532975
3             0.323632           -1.340295               0.532975
4             0.505510           -1.340295               0.532975
```

تبدیل ویژگی‌های عددی به ویژگی‌های فازی(Fuzzy Transformation)

این فرآیند شامل تبدیل ویژگی‌ها به مجموعه‌های فازی است . ابتدا باید توابع فازی را مشخص کنیم:

برای دما و رطوبت دو تابع fuzzy_temperature و fuzzy_humidity تعریف کردیم که بر اساس مقدار هر ویژگی، آن را به یکی از سه دسته"Low"، "Medium"یا "High" اختصاص می‌دهند.

این ویژگی‌ها به نام‌های Fuzzy_Temperature و Fuzzy_Humidity به دیتاست اضافه می‌شوند.

در این قسمت ابتدا با استفاده از دو تابع ویژگی ها را فازی میکنیم و برای قسمت بعدی که ادغام ویژگی است اقدام می کنیم.

ویژگی‌های فازی برای داده‌های نرمال‌شده:

- برای ویژگی‌هایی که با **Min-Max Scaling**نرمال شده‌اند (که به طور معمول در بازه [0, 1] هستند)، بازه‌های فازی به شکل جدیدی تعریف می‌شود:
  - "کم" (کمتر از 0.33)
  - "متوسط" (بین 0.33 و 0.66)
  - "زیاد" (بیشتر از 0.66)

ویژگی‌های فازی برای داده‌های استاندارد شده :برای داده‌هایی که با **Standardization** نرمال شده‌اند (که میانگین 0 و انحراف معیار 1 دارند)، می‌توانیم از بازه‌هایی استفاده کنیم:

- "کمتر از -1"
- "بین -1 و 1"
- "بیشتر از 1"

کد :

```
def fuzzy_temperature(temp):
    if temp < 0.33:
        return 'Low'
    elif 0.33 <= temp < 0.66:
        return 'Medium'
    else:
        return 'High'

def fuzzy_humidity(humidity):
    if humidity < 0.33:
        return 'Low'
    elif 0.33 <= humidity < 0.66:
        return 'Medium'
    else:
        return 'High'
```

برای یک ویژگی دیگر نیز این کار را انجام می دهیم. ویژگی Cooling_Heating_Output_C یکی دیگر از ویژگی هاییست که می تواند در تحلیل ها موثر باشد.

کد:

```
def fuzzy_cooling_heating(output):
    if output < 0.33:
        return 'Low'
    elif 0.33 <= output < 0.66:
        return 'Medium'
    else:
        return 'High'

# اعمال تبدیل فازی به ویژگی `Cooling_Heating_Output_C` (روی داده‌های نرمال‌شده)
scaled_data['Fuzzy_Cooling_Heating_Output'] = scaled_data['Cooling_Heating_Output_C'].apply(fuzzy_cooling_heating)

# نمایش ویژگی جدید فازی `Fuzzy_Cooling_Heating_Output`
print("\nNew fuzzy feature for Cooling_Heating_Output_C:")
print(scaled_data[['Cooling_Heating_Output_C', 'Fuzzy_Cooling_Heating_Output']].head())
```

توضیحات کد:

تعریف تابع fuzzy_cooling_heating : این تابع، مقادیر Cooling_Heating_Output_Cرا به سه دسته "کم"، "متوسط" و "زیاد" تقسیم می‌کند.

- کم :مقادیر کمتر از 0.33
- متوسط :مقادیر بین 0.33 و 0.66
- زیاد :مقادیر بیشتر از 0.66

اعمال تابع فازی به داده‌ها :با استفاده از ()apply این تابع فازی به ستون Cooling_Heating_Output_Cاعمال می‌شود و ستون جدید به نام Fuzzy_Cooling_Heating_Outputبه دیتافریم اضافه می‌شود.

نمایش داده‌ها :داده‌های نهایی شامل ویژگی Cooling_Heating_Output_Cو ویژگی فازی Fuzzy_Cooling_Heating_Outputنمایش داده می‌شوند.

```
New fuzzy feature for Cooling_Heating_Output_C:
   Cooling_Heating_Output_C Fuzzy_Cooling_Heating_Output
0                  0.483984                       Medium
1                  1.301895                         High
2                 -0.242240                          Low
3                  1.358335                         High
4                  0.471683                       Medium
```

تولید ویژگی‌های جدید:

- برخی از ویژگی‌ها می‌توانند با ترکیب داده‌های مختلف تبدیل شوند. به عنوان مثال، ترکیب دما و رطوبت می‌تواند "شرایط محیطی" جدیدی ایجاد کند که برای مدل مفید باشد.

برای این قسمت ویژگی جدیدی به نام Environmental_Condition ایجاد می‌کنیم که ترکیب فازی دما و رطوبت است.

این ویژگی ترکیبی، **یک ویژگی دسته‌ای** (Categorical)خواهد بود که با استفاده از کدگذاری **One-Hot Encoding** تبدیل به ویژگی‌های باینری می‌شود. وقتی ما **Min-Max Scaling**انجام می‌دهیم، ویژگی‌هایی مانند دما و رطوبت به بازه [0, 1] منتقل می‌شوند، و بعد ویژگی‌های فازی طبق این بازه‌ها دسته‌بندی می‌شوند. در نیجه **Environmental_Condition** که ترکیبی از این دو ویژگی فازی است که ممکن است به بازه‌های غیرخطی نیاز داشته باشد. این ویژگی ترکیبی اگر درست کدگذاری نشود، ممکن است با توجه به نحوه دسته‌بندی فازی اشتباه تفسیر شود.

راه حل:

برای اینکه این مشکل به درستی حل شود، می‌توانیم ویژگی **Environmental_Condition**را به طور دقیق‌تری دسته‌بندی کنیم و سپس از **One-Hot Encoding**به‌طور صحیح استفاده کنیم.

کد:

```
scaled_data['Environmental_Condition'] = scaled_data['Fuzzy_Temperature'] + '_' + scaled_data['Fuzzy_Humidity']


encoder = OneHotEncoder(drop='first')  # استفاده می‌کنیم drop='first' برای جلوگیری از کدگذاری بیش از حد از
encoded_conditions = encoder.fit_transform(scaled_data[['Environmental_Condition']]).toarray()

#  One-Hot Encoding
encoded_columns = encoder.categories_[0][1:]
encoded_df = pd.DataFrame(encoded_conditions, columns=encoded_columns)


scaled_data = scaled_data.join(encoded_df)
print("\nData after transforming features (scaled data):")
print(scaled_data.head())
```

توضیح:

کدگذاری :One-Hot با استفاده از **One-Hot Encoding**به ویژگی‌های باینری تبدیل می‌شود.

drop='first' برای جلوگیری از کدگذاری بیش از حد و جلوگیری از هم‌خطی، از drop='first'استفاده کرده‌ایم تا یکی از دسته‌ها را حذف کنیم.

```
Data after transforming features (scaled data):
             Timestamp  Temperature_C  Humidity_%   CO2_ppm  Occupancy_Count  \
0  2017-01-01 00:00:00       0.374639    0.373570  0.730058         0.244898
1  2017-01-01 00:10:00       0.950982    0.332832  0.184491         0.530612
2  2017-01-01 00:20:00       0.732198    0.176037  0.346643         0.755102
3  2017-01-01 00:30:00       0.598823    0.607250  0.663330         0.653061
4  2017-01-01 00:40:00       0.156053    0.476577  0.482112         0.102041

   External_Temperature_C        Kp        Ki        Kd  \
0                0.091693  0.954611 -1.688958  1.371281
1                0.323699 -0.888653 -1.337428 -0.625012
2                0.880285 -1.119617  1.383654  0.437612
3                0.712079 -0.747488  1.278454  0.240834
4                0.785969  1.449495  1.128618  1.679523

   Fuzzy_Adjustment_Factor  ...  Fuzzy_Humidity  Environmental_Condition  \
0                -1.303457  ...          Medium            Medium_Medium
1                 0.490879  ...          Medium              High_Medium
2                -0.079904  ...             Low                High_Low
3                -1.662421  ...          Medium            Medium_Medium
4                 1.704645  ...          Medium               Low_Medium

   High_Low  High_Medium  Low_High  Low_Low  Low_Medium Medium_High  \
0       0.0          0.0       0.0      0.0         0.0         0.0
1       0.0          1.0       0.0      0.0         0.0         0.0
2       1.0          0.0       0.0      0.0         0.0         0.0
3       0.0          0.0       0.0      0.0         0.0         0.0
4       0.0          0.0       0.0      0.0         1.0         0.0

  Medium_Low Medium_Medium
0        0.0           1.0
1        0.0           0.0
2        0.0           0.0
3        0.0           1.0
4        0.0           0.0
```

توضیحات ستون‌های جدید:

1. Environmental_Condition این ویژگی ترکیبی است که از ترکیب دو ویژگی فازی "دما" و "رطوبت" به دست آمده است. به طور مثال، اگر دما "کم" و رطوبت "کم" باشد، این ویژگی به مقدار "Low_Low" خواهد بود.
2. High_Low نشان‌دهنده ترکیب فازی "High"برای دما و "Low"برای رطوبت است.
   - مقدار 1به این معناست که دما "زیاد" و رطوبت "کم" است.
   - مقدار 0نشان‌دهنده عدم تطابق این ترکیب است.
3. High_Medium نشان‌دهنده ترکیب فازی "High"برای دما و "Medium"برای رطوبت است.
   - مقدار 1به این معناست که دما "زیاد" و رطوبت "متوسط" است.
4. Low_High نشان‌دهنده ترکیب فازی "Low"برای دما و "High"برای رطوبت است.
   - مقدار 1به این معناست که دما "کم" و رطوبت "زیاد" است.
5. Low_Low نشان‌دهنده ترکیب فازی "Low"برای دما و "Low"برای رطوبت است.
   - مقدار 1به این معناست که دما "کم" و رطوبت "کم" است.
6. Low_Medium نشان‌دهنده ترکیب فازی "Low"برای دما و "Medium"برای رطوبت است.
   - مقدار 1به این معناست که دما "کم" و رطوبت "متوسط" است.
7. Medium_High نشان‌دهنده ترکیب فازی "Medium"برای دما و "High"برای رطوبت است.
   - مقدار 1به این معناست که دما "متوسط" و رطوبت "زیاد" است.
8. Medium_Low نشان‌دهنده ترکیب فازی "Medium"برای دما و "Low"برای رطوبت است.
   - مقدار 1به این معناست که دما "متوسط" و رطوبت "کم" است.
9. Medium_Medium نشان‌دهنده ترکیب فازی "Medium"برای دما و "Medium"برای رطوبت است.
   - مقدار 1به این معناست که دما "متوسط" و رطوبت "متوسط" است.

ویژگی‌های باینری یا کدگذاری دسته‌ای:

- **HVAC_Efficiency_Class** که مقادیر آن بین 0 و 2 است، می‌تواند به ویژگی‌های باینری یا کدگذاری دسته‌ای تبدیل شود.

چون این ویژگی دارای سه مقدار ممکن (0، 1، و 2) است، می‌توانیم آن را به دو روش مختلف تبدیل کنیم:

1. کدگذاری دسته‌ای :(One-Hot Encoding) که ویژگی HVAC_Efficiency_Classرا به سه ویژگی باینری تبدیل می‌کند.
2. کدگذاری عددی :(Label Encoding) که مقادیر 0، 1 و 2 را به اعداد متناسب تبدیل می‌کند. اما این روش معمولاً وقتی ویژگی‌ها به صورت ترتیبی هستند، مناسب است.

برای این منظور، از One-Hot Encodingاستفاده خواهیم کرد تا هر یک از مقادیر این ویژگی به یک ستون جداگانه با مقدار 0 یا 1 تبدیل شود

خروجی :

```
Data after transforming HVAC_Efficiency_Class to One-Hot Encoding:
            Timestamp   Temperature_C   Humidity_%   CO2_ppm  \
0  2017-01-01 00:00:00        0.374639     0.373570  0.730058
1  2017-01-01 00:10:00        0.950982     0.332832  0.184491
2  2017-01-01 00:20:00        0.732198     0.176037  0.346643
3  2017-01-01 00:30:00        0.598823     0.607250  0.663330
4  2017-01-01 00:40:00        0.156053     0.476577  0.482112

    Occupancy_Count   External_Temperature_C        Kp        Ki        Kd  \
0          0.244898                0.091693  0.954611 -1.688958  1.371281
1          0.530612                0.323699 -0.888653 -1.337428 -0.625012
2          0.755102                0.880285 -1.119617  1.383654  0.437612
3          0.653061                0.712079 -0.747488  1.278454  0.240834
4          0.102041                0.785969  1.449495  1.128618  1.679523

    Fuzzy_Adjustment_Factor  ...   High_Low   High_Medium   Low_High  \
0                -1.303457  ...        0.0           0.0        0.0
1                 0.490879  ...        0.0           1.0        0.0
2                -0.079904  ...        1.0           0.0        0.0
3                -1.662421  ...        0.0           0.0        0.0
4                 1.704645  ...        0.0           0.0        0.0

    Low_Low   Low_Medium   Medium_High   Medium_Low  Medium_Medium -0.85462  \
0       0.0          0.0           0.0          0.0            1.0      0.0
1       0.0          0.0           0.0          0.0            0.0      0.0
2       0.0          0.0           0.0          0.0            0.0      0.0
3       0.0          0.0           0.0          0.0            1.0      0.0
4       0.0          1.0           0.0          0.0            0.0      0.0

   0.532975
0       1.0
1       1.0
2       1.0
3       1.0
4       1.0
```

ممبرشیپ فانکشن هر ویژگی باید با توجه به این بازه بندی ها صورت بگیرد هر کدام از ممبرشیپ فانکشن های مثلثی را در یک تابع معرفی میکنیم.

برای دما:(Temperature_C)

- Lowمقدار عضویت بالاتر برای دماهای پایین (مثلاً 0 تا 20 درجه)
- Mediumبرای دماهای متوسط (مثلاً بین 20 و 27 درجه)
- Highبرای دماهای بالا (مثلاً 27 تاu 40 درجه)

برای رطوبت:(Humidity_%)

- :Lowرطوبت پایین (مثلاً کمتر از 40٪)
- :Mediumرطوبت متوسط (مثلاً بین 40٪ و 60٪)
- :Highرطوبت بالا (مثلاً بالاتر از 60٪)

برای خروجی سرمایشی/گرمایشی:(Cooling_Heating_Output_C)

- :Lowخروجی کم (مثلاً 0 تا 0.33)
- :Mediumخروجی متوسط (مثلاً بین 0.33 و 0.66)
- :Highخروجی زیاد (مثلاً بالاتر از 0.66)

```python
def membership_temperature_low(x):
    return np.maximum(0, np.minimum(1, (0.33 - x) / 0.33))

def membership_temperature_medium(x):
    return np.maximum(0, np.minimum(1, 1 - abs(x - 0.5) / 0.33))

def membership_temperature_high(x):
    return np.maximum(0, np.minimum(1, (x - 0.66) / 0.33))

# تابع عضویت مثلثی برای رطوبت (نرمال‌شده)
def membership_humidity_low(x):
    return np.maximum(0, np.minimum(1, (0.33 - x) / 0.33))

def membership_humidity_medium(x):
    return np.maximum(0, np.minimum(1, 1 - abs(x - 0.5) / 0.33))

def membership_humidity_high(x):
    return np.maximum(0, np.minimum(1, (x - 0.66) / 0.33))

# تابع عضویت مثلثی برای خروجی سرمایشی/گرمایشی (نرمال‌شده)
def membership_cooling_heating_low(x):
    return np.maximum(0, np.minimum(1, (0.33 - x) / 0.33))

def membership_cooling_heating_medium(x):
    return np.maximum(0, np.minimum(1, 1 - abs(x - 0.5) / 0.33))

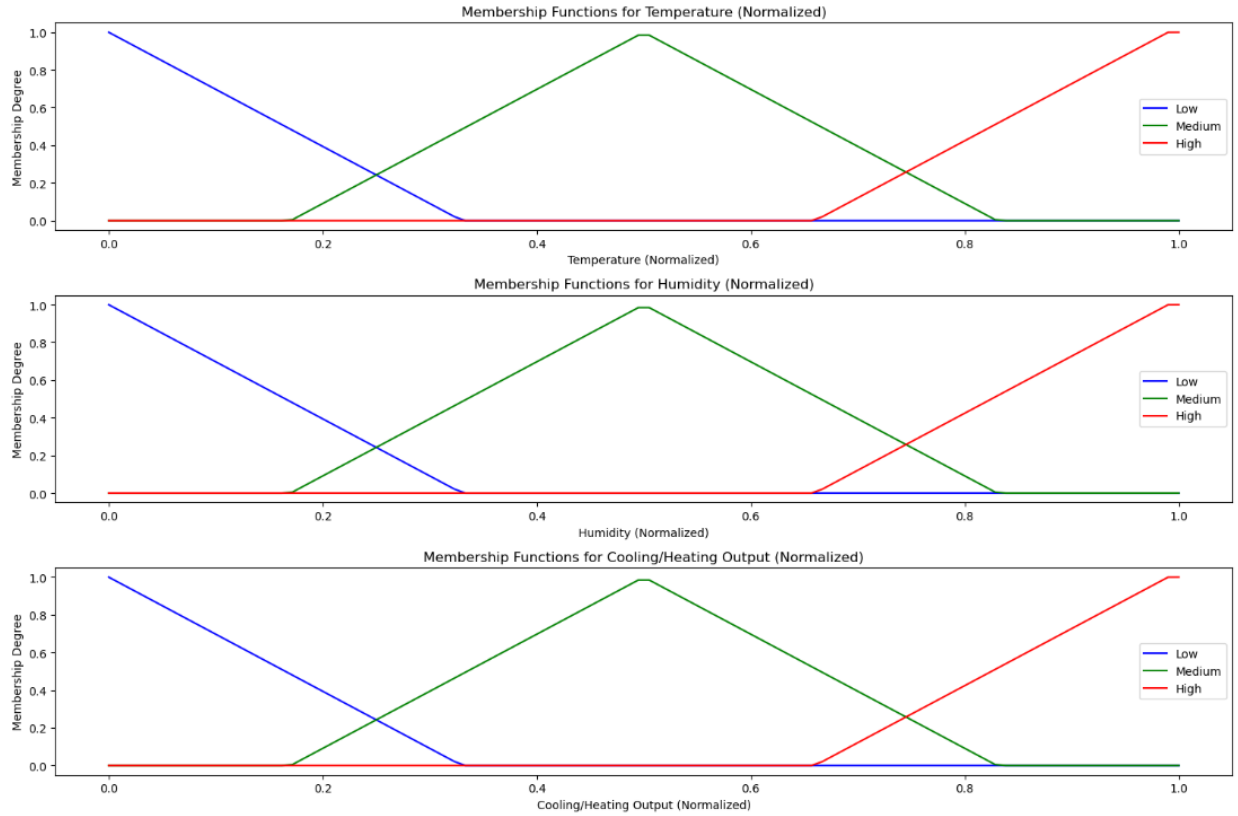
def membership_cooling_heating_high(x):
    return np.maximum(0, np.minimum(1, (x - 0.66) / 0.33))

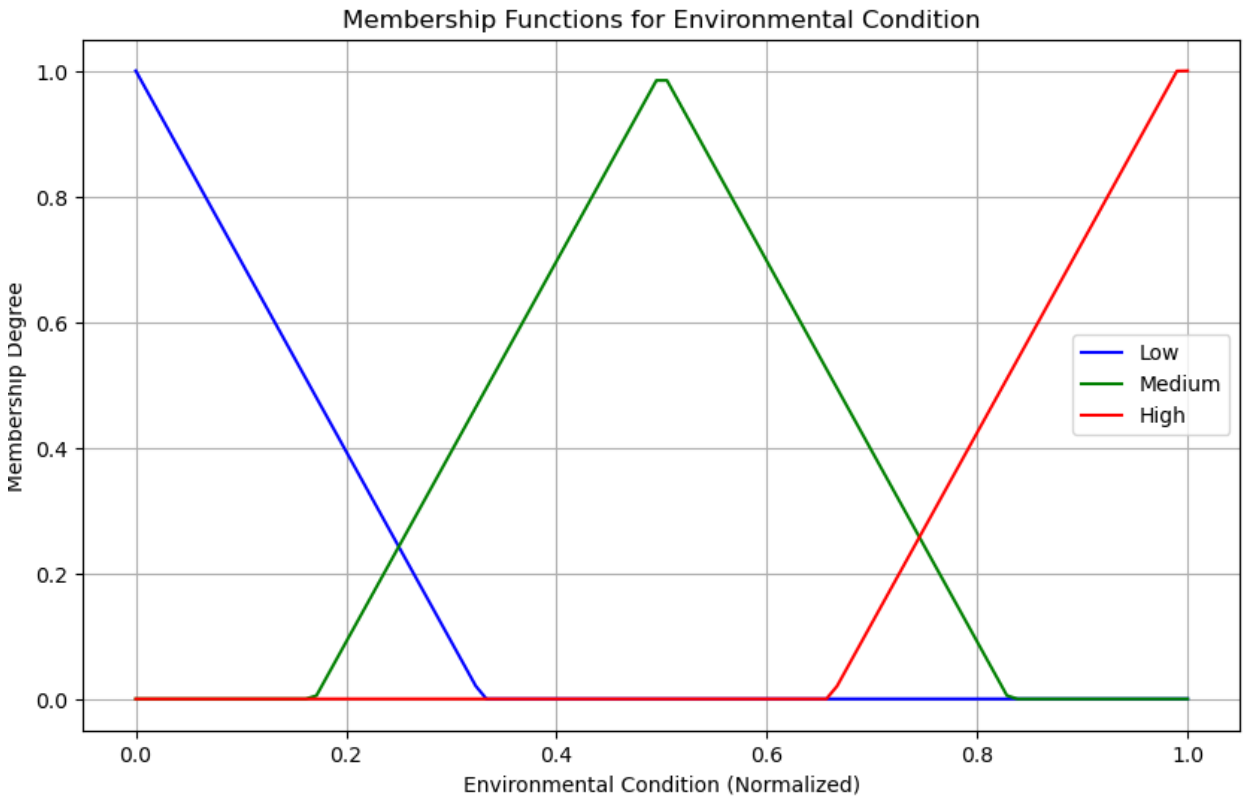
# تابع عضویت برای ویژگی ترکیبی Environmental_Condition
def membership_env_condition_low(x):
    return np.minimum(membership_temperature_low(x), membership_humidity_low(x))

def membership_env_condition_medium(x):
    return np.minimum(membership_temperature_medium(x), membership_humidity_medium(x))

def membership_env_condition_high(x):
    return np.minimum(membership_temperature_high(x), membership_humidity_high(x))
```

سپس با توجه به subplot هر کدام را رسم میکنیم. چهار نمودار مثلثی داریم که نمایش انها به این صورت می شود:





حالا باید برای **مصرف انرژی** (به عنوان خروجی) نیز توابع عضویت مشابهی تعریف کنیم.

برای مصرف انرژی، می‌توانیم سه دسته‌ی کم، متوسط و زیاد را در نظر بگیریم که مطابق با تغییرات مختلف دما، رطوبت، و خروجی سرمایشی/گرمایشی، مصرف انرژی افزایش یا کاهش یابد. در اینجا نیز توابع عضویت به صورت مثلثی مطرح شده است. هر تابع قله و مینیمم های محاسبه شده را گزارش میدهد تا با وارد شدن عدد بتواند میزان عضویت را بررسی بنماید.

```
def membership_energy_low(x):
    return np.maximum(0, np.minimum(1, (0.33 - x) / 0.33))

def membership_energy_medium(x):
    return np.maximum(0, np.minimum(1, 1 - abs(x - 0.5) / 0.33))

def membership_energy_high(x):
    return np.maximum(0, np.minimum(1, (x - 0.66) / 0.33))
```

تعریف قوانین استنتاج(Rule Inference)

تا اینجا ما ورودی ها را تعیین کردیم و فازی سازیشان کردیم، توابع عضویت یا ممبرشیپ فانکشن ها را هم تعیین کردیم الان زمان تعیین قوانین است. از آنجایی که

در تابع def fuzzy_inference قوانین را توضیح می دهیم یعنی ابتدا ورودی ها را تعیین کرده و سپس به تعاریف قوانین میپردازیم.

.ورودی‌ها:

- دما :(Temperature) تأثیر مستقیم بر مصرف انرژی دارد. دماهای بالاتر معمولاً نیاز به سرمایش بیشتر دارند.
- رطوبت: تأثیر بر مصرف انرژی دارد، زیرا کنترل رطوبت هم‌زمان با دما نیازمند مصرف انرژی بیشتر است.
- خروجی سرمایشی/گرمایشی :(Cooling/Heating Output) اگر این مقدار بالا باشد، نیاز به انرژی بیشتری برای سرمایش یا گرمایش خواهد بود.
- HVAC (HVAC Efficiency) سیستم‌های با کارایی بالاتر مصرف انرژی کمتری دارند.

کد:

```
temp_low = membership_temperature_low(temp)
temp_medium = membership_temperature_medium(temp)
temp_high = membership_temperature_high(temp)

humidity_low = membership_humidity_low(humidity)
humidity_medium = membership_humidity_medium(humidity)
humidity_high = membership_humidity_high(humidity)

cooling_low = membership_cooling_heating_low(cooling)
cooling_medium = membership_cooling_heating_medium(cooling)
cooling_high = membership_cooling_heating_high(cooling)

hvac_efficiency_low = membership_cooling_heating_low(hvac_efficiency)
hvac_efficiency_medium = membership_cooling_heating_medium(hvac_efficiency)
hvac_efficiency_high = membership_cooling_heating_high(hvac_efficiency)
```

در اینجا عضویت هر ورودی تعیین می شود با استفاده از توابعی که در بخش های قبل معین کرده بودیم. سپس باید برای خروجی

قوانین پیشنهادی:

- قانون 1 :اگر دما "زیاد" و رطوبت "کم" باشد، مصرف انرژی "زیاد" خواهد بود.
- قانون 2 :اگر دما "زیاد" و رطوبت "زیاد" باشد، مصرف انرژی "زیاد" خواهد بود.

- قانون 3 :اگر دما "متوسط" و رطوبت "کم" باشد، مصرف انرژی "متوسط" خواهد بود.
- قانون 4 :اگر دما "متوسط" و رطوبت "متوسط" باشد، مصرف انرژی "متوسط" خواهد بود.
- قانون 5 :اگر دما "کم" و رطوبت "کم" باشد، مصرف انرژی "کم" خواهد بود.
- قانون 6 :اگر خروجی سرمایشی/گرمایشی "زیاد" باشد، مصرف انرژی "زیاد" خواهد بود.
- قانون 7 :اگر کارایی" HVAC بالا" باشد، مصرف انرژی "کم" خواهد بود.

کد مربوط به این بخش :

```
# قوانین فازی برای پیش‌بینی مصرف انرژی
rule1 = min(temp_high, humidity_low)  # دما بالا و رطوبت کم: مصرف انرژی زیاد
rule2 = min(temp_high, humidity_high)  # دما بالا و رطوبت زیاد: مصرف انرژی خیلی زیاد
rule3 = min(temp_medium, humidity_medium)  # دما متوسط و رطوبت متوسط: مصرف انرژی متوسط
rule4 = min(temp_low, humidity_low)  # دما کم و رطوبت کم: مصرف انرژی کم
rule5 = min(temp_high, cooling_high)  # دما بالا و خروجی سرمایشی زیاد: مصرف انرژی خیلی زیاد
rule6 = hvac_efficiency_high  # کارایی HVAC کم :بالا: مصرف انرژی
rule7 = min(cooling_high, hvac_efficiency_low)  # خروجی سرمایشی زیاد و کارایی HVAC زیاد :پایین: مصرف انرژی
```

سپس برای خروجی سه حالت تعیین می کنیم:

energy_low = min(rule4, rule6)

energy_medium = max(rule3, min(rule1, rule2))

energy_high = max(rule5, rule7)

DeFuzzification :

حال برای آنکه دیفازیفیکیشن صورت بگیرد از این خط استفاده می کنیم تا برای مصرف انرژی یک عدد نسبت بدهیم:

energy = (energy_low * 0 + energy_medium * 0.5 + energy_high * 1) / (energy_low + energy_medium + energy_high)

بررسی عملکرد سیستم فازی طراحی شده

درجه انطباق سیستم:

برای هر قانون و ترکیب فازی که اعمال می‌شود، ما می‌توانیم یک درجه انطباق یا کارایی برای سیستم محاسبه کنیم که نشان دهد چقدر پیش‌بینی‌های سیستم با وضعیت واقعی تطابق دارند. این کار را با محاسبه عضویت کلی برای هر ترکیب ورودی و خروجی انجام می‌دهیم و از آن برای ارزیابی مدل استفاده می‌کنیم.

```
    # محاسبه درجه انطباق
    degree_of_fuzziness = (energy_low + energy_medium + energy_high) / 3
    return energy, degree_of_fuzziness


temp = 0.7  # دما (در بازه [0, 1])
humidity = 0.4  # رطوبت (در بازه [0, 1])
cooling = 0.5  # خروجی سرمایشی/گرمایشی (در بازه [0, 1])
hvac_efficiency = 0.8  # کارایی HVAC (در بازه [0, 1])

predicted_energy, degree_of_fuzziness = fuzzy_inference(temp, humidity, cooling, hvac_efficiency)
```

نتیجه نهایی این مدل سازی با فازی:

Degree of Fuzziness (System Fit): 0.13

این عدد خیلی دلخواه نیست، درجه فازی شدگی از 0.2 تا 0.4 بازه ای است که نسبتا مدل سازی قابل قبول است اما این عدد از این بازه نیز کمتر است بنابراین به فکر بهینه سازی می افتیم.

بهینه سازی1:

به جای تعریف توابع عضویت به شکل مثلثاتی به شکل ذوزنقه ای تعریف می کنیم:

```
def membership_temperature_low(x):
    if x < 0.33:
        return 1
    elif 0.33 <= x < 0.66:
        return (0.66 - x) / 0.33
    else:
        return 0

def membership_temperature_medium(x):
    if 0.33 <= x < 0.66:
        return (x - 0.33) / 0.33
    elif 0.66 <= x <= 1:
        return (1 - x) / 0.33
    else:
        return 0

def membership_temperature_high(x):
    if x > 0.66:
        return 1
    elif 0.33 < x <= 0.66:
        return (x - 0.33) / 0.33
    else:
        return 0
def membership_humidity_low(x):
    if x < 0.33:
        return 1
    elif 0.33 <= x < 0.66:
        return (0.66 - x) / 0.33
    else:
        return 0

def membership_humidity_medium(x):
    if 0.33 <= x < 0.66:
        return (x - 0.33) / 0.33
    elif 0.66 <= x <= 1:
        return (1 - x) / 0.33
    else:
        return 0

def membership_humidity_high(x):
    if x > 0.66:
        return 1
    elif 0.33 < x <= 0.66:
        return (x - 0.33) / 0.33
    else:
        return 0

def membership_cooling_heating_low(x):
    if x < 0.33:
        return 1
    elif 0.33 <= x < 0.66:
        return (0.66 - x) / 0.33
    else:
        return 0
```

این قسمتی از تعاریف ذوزنقه ایست سپس همه کار های قبل را انجام داده و خروجی میگیریم و خروجی به این شکل می شود:

Degree of Fuzziness (System Fit): 0.2424242424242424

بهینه سازی 2:

در قسمت بعد بهینه سازی تصمیم برآن شد که روش تجمیع قوانین و محاسبه عدد نهایی ینی عددی سازی سیستم فازی (دی فازیفیکیشن) را به نحوی دیگر بررسی کنیم.

در این قسمت از روش Weighted Average استفاده کردیم و نتیجه به این صورت شد:

Rule	ترکیب ورودی	توضیح معنایی	مقدار خروجی (Weight)	معنی وزن
1	Temp High و Humidity Low	دما بالا، رطوبت کم	0.8	نشان‌دهنده مصرف نسبتا زیاد انرژی
2	Temp High و Humidity High	دما بالا، رطوبت زیاد	1	نشان‌دهنده مصرف خیلی زیاد انرژی
3	Temp Medium و Humidity Medium	دما و رطوبت متوسط	0.5	نشان‌دهنده مصرف متوسط انرژی
4	Temp Low و Humidity Low	دما و رطوبت کم	0	نشان‌دهنده مصرف کم انرژی
5	Temp High و Cooling High	دما بالا و سرمایش زیاد	1	نشان‌دهنده مصرف زیاد انرژی
6	HVAC Efficiency High	کارایی HVAC بالا	0.15	نشان‌دهنده مصرف کم انرژی به دلیل بهره‌وری سیستم
7	Cooling High و HVAC Efficiency Low	سرمایش زیاد و کارایی پایین	1	نشان‌دهنده مصرف زیاد انرژی

کد:

```
total_weight = rule1 + rule2 + rule3 + rule4 + rule5 + rule6 + rule7

if total_weight == 0:
    energy = 0
else:
    energy = (rule1 * 0.8 + rule2 * 1 + rule3 * 0.5 + rule4 * 0 + rule5 * 1 + rule6 * 0.15 + rule7 * 1) / total_weight

adjusted_energy = energy * fuzzy_adjustment_factor


degree_of_fuzziness = total_weight / 7

return adjusted_energy, degree_of_fuzziness
```

توضیحات کد: وزن هر Rule نشان می‌دهد که در صورت فعال بودن آن قانون، چه مقدار انرژی باید مصرف شود.

عدد بزرگتر ⟶ مصرف بیشتر انرژی.

عدد کوچکتر ⟶ مصرف کمتر انرژی.

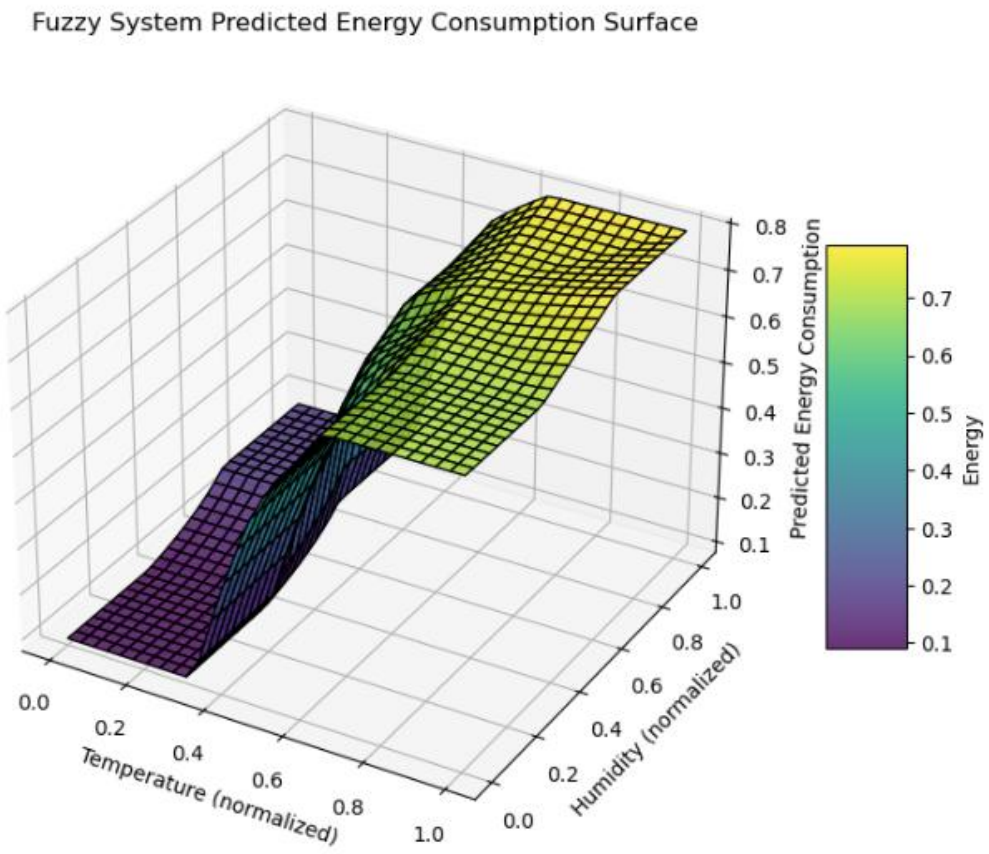
این اعداد به مرحله Defuzzificationمنتقل می‌شوند تا خروجی نهایی سیستم فازی محاسبه شود.

ترکیب میزان فعال بودن Rule × وزن Rule در میانگین وزنی، تاثیر هر قانون بر نتیجه نهایی را مشخص می‌کند.

در نهایت خروجی این سیستم با همین قوانین به این شکل در می آید که خوشحال کننده است:

```
Predicted Energy Consumption: 0.5299411764705882
Degree of Fuzziness (System Fit): 0.36796536796536794
```

رسم نمودار برای این خروجی :



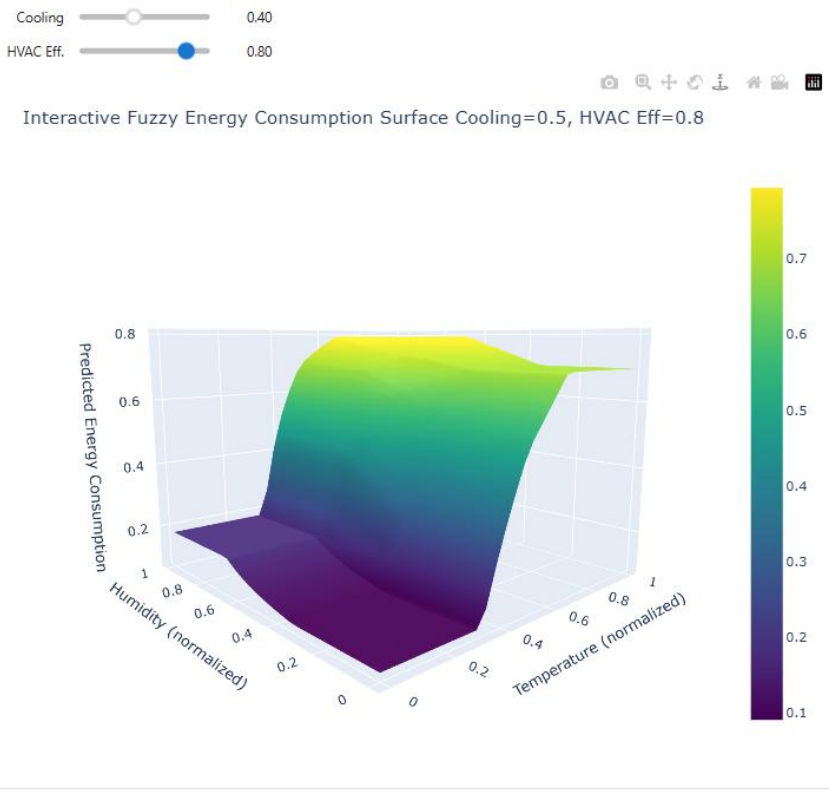
توضیح نمودار

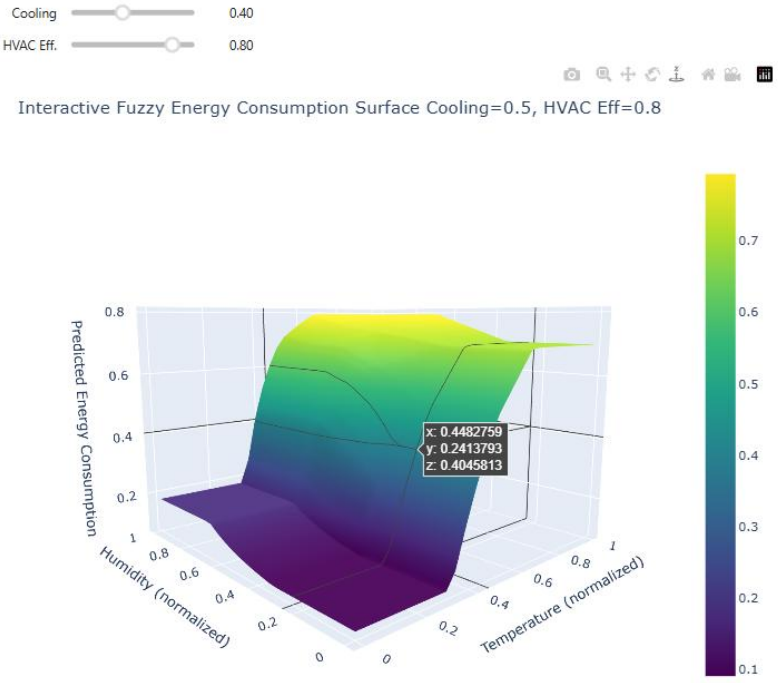
- محور X: دما نرمال‌شده (0–1)
- محور Y: رطوبت نرمال‌شده (0–1)
- محور Z: مصرف انرژی پیش‌بینی‌شده توسط سیستم فازی
- رنگ‌ها: نشان‌دهنده میزان انرژی مصرفی هستند.

با این نمودار می‌توان دید که چگونه ترکیب دما و رطوبت، خروجی سیستم فازی را تغییر می‌دهد و کدام مناطق باعث مصرف زیاد یا کم انرژی می‌شوند.

خلاقیت:

در ادامه این نمودار را به صورت INTERACTIVE طراحی کردیم که باعث میشود بتوانیم متغیر ها را تغییر داده و نمودار نهایی که تغییر میکند را به صورت زنده مشاهده کنیم.


Cooling
0.40
HVAC Eff.
0.80
Interactive Fuzzy Energy Consumption Surface Cooling=0.5, HVAC Eff=0.8
Predicted Energy Consumption
Humidity (normalized)
Temperature (normalized)


Cooling
0.40
HVAC Eff.
0.80
Interactive Fuzzy Energy Consumption Surface Cooling=0.5, HVAC Eff=0.8
Predicted Energy Consumption
Humidity (normalized)
Temperature (normalized)
x: 0.4482759
y: 0.2413793
z: 0.4045813

بررسی سیستم فازی

در ادامه برای اینکه سیستم طراحی شده را بررسی کنیم و نقاط ضعف و قوت آنرا استخراج کنیم از این مراحل استفاده کردیم:

۱ .تعریف قوانین(Rules)

در ابتدا یک لیست از قوانین سیستم فازی تعریف شده است. هر قانون شامل سه بخش است:

1. وضعیت دما(Temperature) → High ، Medium، Low یاAny
2. وضعیت رطوبت(Humidity) → High ، Medium، Low یاAny
3. خروجی مصرف انرژی(Energy) → Energy_Low ، Energy_Medium، Energy_High، Energy_VeryHigh

این قوانین به زبان ساده می‌گویند که بر اساس ترکیب دما و رطوبت، مصرف انرژی چقدر خواهد بود.

- به عنوان مثال: "اگر دما High و رطوبت Low باشد، مصرف انرژی High است."
- اگر از Any استفاده شود، یعنی پیش‌شرط مستقل از آن متغیر است؛ به عبارتی آن متغیر اهمیتی ندارد.
- این ساختار برای سیستم‌های فازی استاندارد است، زیرا فازی‌ها با دسته‌بندی‌های انسانی (Low, Medium, High) کار می‌کنند و لازم نیست تمام مقادیر دقیق ورودی را پوشش دهند.

۲ .شمارش تعداد قوانین

کد:

```
# 1. تعداد قوانین
num_rules = len(rules)
print(f"Number of rules in the fuzzy system: {num_rules}")
```

کد تعداد قوانین را می‌شمارد و چاپ می‌کند.

دلیل استفاده:

- اطلاع از تعداد قوانین به شما کمک می‌کند بفهمید چقدر فضای تصمیم سیستم فازی توسط قوانین پوشش داده شده است.
- هرچه قوانین زیاد و پراکنده باشند، سیستم پیچیده‌تر ولی دقیق‌تر است؛ قوانین کمتر ساده‌تر ولی ممکن است پوشش ناقص داشته باشند.

۳ .بررسی تضاد قوانین(Conflict Detection)

کد:

```
def check_rule_conflicts(rules):
    conflicts = []
    seen_conditions = {}
    for rule in rules:
        conditions = tuple(rule[:-1])  # پیش‌شرط‌ها
        output = rule[-1]
        if conditions in seen_conditions:
            if seen_conditions[conditions] != output:
                conflicts.append((conditions, seen_conditions[conditions], output))
        else:
            seen_conditions[conditions] = output
    return conflicts

conflicts = check_rule_conflicts(rules)
if conflicts:
    print("Conflicting rules detected:")
    for c in conflicts:
        print(f"Conditions: {c[0]}, Outputs: {c[1]} vs {c[2]}")
else:
    print("No conflicts detected in the rules.")
```

در این بخش، سیستم بررسی می‌کند که آیا دو قانون با همان پیش‌شرط‌ها، خروجی متفاوت دارند یا نه.

مکانیزم کار:

- یک دیکشنری ایجاد می‌شود تا پیش‌شرط‌های هر قانون را ذخیره کند.
- اگر قانون جدیدی پیدا شود که پیش‌شرط‌هایش با یک قانون قبلی یکسان باشد ولی خروجی متفاوت داشته باشد، این تضاد (Conflict) ثبت می‌شود.

دلیل استفاده:

- تضاد قوانین می‌تواند باعث رفتار غیرمنطقی سیستم فازی شود، زیرا یک ترکیب ورودی نمی‌تواند همزمان دو خروجی متفاوت داشته باشد.
- بررسی تضاد، کیفیت قوانین را بالا می‌برد و سیستم را پایدارتر می‌کند.

۴ .بررسی پوشش قوانین(Coverage)

کد:

```
def compute_rule_coverage(rules, input_variables):
    covered = 0
    total = len(input_variables[0]) * len(input_variables[1])  # ترکیب تمام ورودی‌ها
    for temp_val in input_variables[0]:
        for hum_val in input_variables[1]:
            rule_covered = False
            for rule in rules:
                temp_rule = rule[0]
                hum_rule = rule[1]
                # بررسی اینکه آیا پیش‌شرط‌ها با ورودی‌های مختلف تطابق دارند
                if (temp_rule == 'Any' or temp_rule == temp_val) and \
                   (hum_rule == 'Any' or hum_rule == hum_val):
                    covered += 1
                    rule_covered = True
                    break  # اگر یک قانون برای ترکیب ورودی‌ها پیدا شد، ادامه نده
    coverage_percent = (covered / total) * 100
    return coverage_percent
```

این بخش بررسی می‌کند که چقدر فضای ورودی‌ها توسط قوانین پوشش داده شده است.

- ابتدا ترکیب تمام مقادیر دما و رطوبت ساخته می‌شود.
- برای هر ترکیب، بررسی می‌شود که آیا یک قانون موجود است که آن ترکیب را پوشش دهد یا خیر.
- اگر پیش‌شرط‌ها Any باشند، آن قانون تقریباً همه ترکیب‌های ممکن را شامل می‌شود.
- در نهایت درصد پوشش محاسبه و چاپ می‌شود.

دلیل استفاده:

- پوشش ناکامل قوانین یعنی سیستم ممکن است برای بعضی ورودی‌ها خروجی نداشته باشد یا رفتار پیش‌بینی‌نشده‌ای داشته باشد.
- بررسی پوشش کمک می‌کند اطمینان حاصل شود که تقریباً تمام حالت‌های ورودی تحت پوشش سیستم فازی هستند.

در نهایت پس از ران کردن کد این بخش متوجه می شویم که:

- تعداد قوانین سیستم را چند تاست.
- تضادهای احتمالی بین قوانین را شناسایی و برای رفع آنها اقدام کنیم.
- اطمینان حاصل کنیم که تمام فضای ورودی‌ها (Temperature × Humidity) تحت پوشش قوانین است.

خروجی کد:

```
Number of rules in the fuzzy system: 10
No conflicts detected in the rules.
Updated Rule coverage (input space covered by rules): 100.00%
```

این خروجی خوشحال کننده است زیرا همه فضا توسط قوانین پوشانده شده و هیچ تضادی در قوانین وجود ندارد.

در ادامه برای جلوه بصری این بخش را به صورت نمودار پدید می اوریم تا ببینیم ایا محلی از فضای سیستم هست که توسط قوانین پوشش داده نشده یا خیر:

کد:

```
# فضای ورودی‌ها
temp_space = ['Low', 'Medium', 'High']
humidity_space = ['Low', 'Medium', 'High']

# ایجاد ماتریس پوشش: 1 = تحت پوشش، 0 = بدون پوشش
coverage_matrix = np.zeros((len(humidity_space), len(temp_space)))

for i, hum_val in enumerate(humidity_space):
    for j, temp_val in enumerate(temp_space):
        for rule in rules:
            temp_rule, hum_rule, _ = rule
            if (temp_rule == 'Any' or temp_rule == temp_val) and \
               (hum_rule == 'Any' or hum_rule == hum_val):
                coverage_matrix[i, j] = 1  # این ترکیب ورودی تحت پوشش است
                break  # اگر یک قانون پوشش داد، ادامه نده

# رسم نمودار پوشش
fig, ax = plt.subplots(figsize=(6,5))
cax = ax.matshow(coverage_matrix, cmap='Greens', vmin=0, vmax=1)

# برچسب محورها
ax.set_xticks(range(len(temp_space)))
ax.set_xticklabels(temp_space)
ax.set_yticks(range(len(humidity_space)))
ax.set_yticklabels(humidity_space)

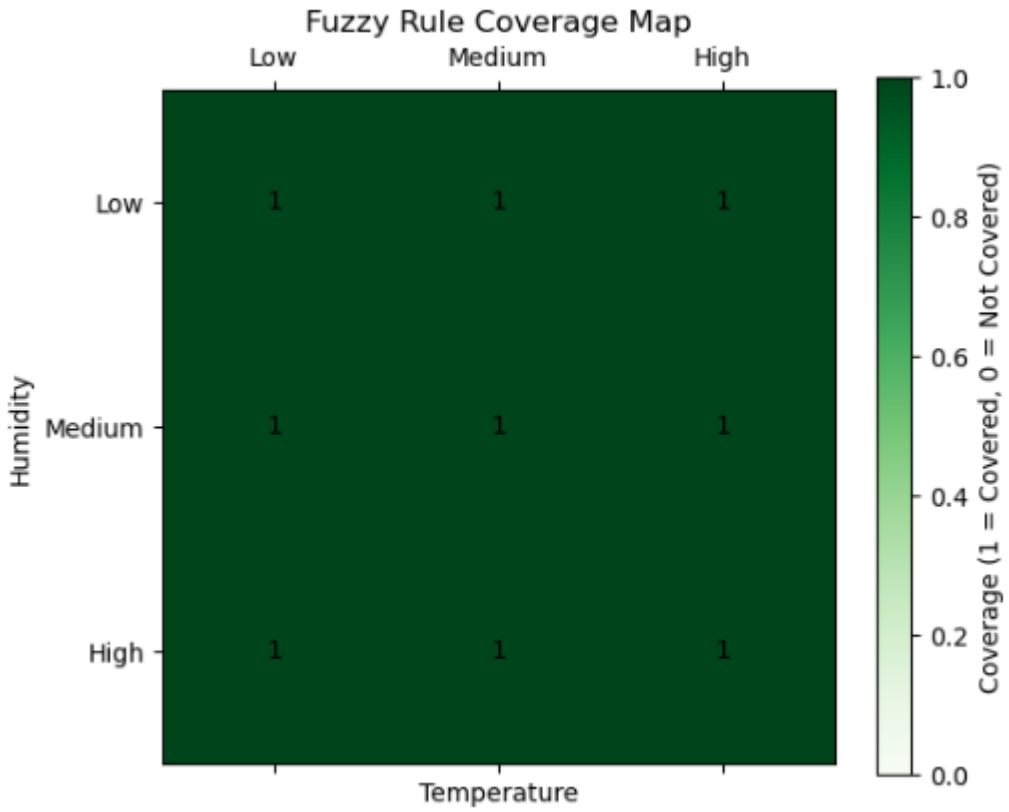
# عنوان و رنگ‌بندی
plt.title('Fuzzy Rule Coverage Map')
fig.colorbar(cax, label='Coverage (1 = Covered, 0 = Not Covered)')

# نمایش مقادیر داخل هر خانه
for i in range(len(humidity_space)):
    for j in range(len(temp_space)):
        ax.text(j, i, int(coverage_matrix[i, j]), ha='center', va='center', color='black')

plt.xlabel('Temperature')
plt.ylabel('Humidity')
plt.show()
```

دلیل/هدف	کاری که انجام می‌دهد	بخش کد
مشخص کردن همه ترکیب‌های ورودی سیستم فازی	تعیین مقادیر ممکن برای دما و رطوبت	*تعریف فضای ورودی‌ها (temp_space و humidity_space)*
هر خانه = 0 (پوشش ندارد) یا 1 (پوشش داده شده)	ساخت ماتریس صفر برای نگهداری وضعیت پوشش	*ایجاد ماتریس پوشش (coverage_matrix = np.zeros(...))*
اطمینان از اینکه هر ترکیب ورودی حداقل یک قانون دارد	برای هر ترکیب دما و رطوبت، بررسی می‌شود که آیا یک قانون آن را پوشش می‌دهد یا خیر	*پر کردن ماتریس پوشش (حلقه‌ها و بررسی قوانین)*
دیداری کردن پوشش قوانین	ایجاد Heatmap از ماتریس پوشش با رنگ سبز برای پوشش	*رسم نمودار ماتریسی (ax.matshow)*
خوانایی نمودار و فهم سریع محورها	محور X = دما، محور Y = رطوبت و برچسب‌گذاری مقادیر	*برچسب محورها (set_xticks, set_yticks)*
نشان دادن مقادیر پوشش و توضیح نمودار	گذاشتن عنوان و اضافه کردن Colorbar	*عنوان و رنگ‌بندی (plt.title, fig.colorbar)*
نشان دادن وضعیت پوشش هر ترکیب به صورت عددی	نوشتن ۰ یا ۱ داخل هر خانه	*نمایش مقادیر داخل خانه‌ها (ax.text)*
کامل کردن نمودار و ارائه بصری پوشش قوانین	مشخص کردن نام محورها و نمایش نمودار	*برچسب محورها و نمایش نهایی (plt.xlabel, plt.ylabel, plt.show)*

خروجی :



توضیح نمودار:

- محور X: دما(Low, Medium, High)
- محور Y: رطوبت(Low, Medium, High)
- رنگ خانه‌ها:
  - سبز (1) ⟵ این ترکیب ورودی توسط حداقل یک قانون پوشش داده شده است
  - سفید (0) ⟵ این ترکیب پوشش داده نشده است
- عدد داخل هر خانه هم نشان‌دهنده وضعیت پوشش (0 یا 1) است.

با این نمودار به راحتی می‌توانید ببینید که تمام فضای ورودی‌ها تحت پوشش قوانین سیستم فازی هستند یا خیر و اگر خانه‌ای بدون پوشش باشد، نیاز به افزودن قانون جدید است.

در نتیجه می توانیم بگوییم این مدل فازی ما برای پیش بینی مصارف انرژی خوب کار کرده و میتواند برای سیستم های کنترلی در مکان هایی که سنسور های اندازه گیری مطابق با ورودی های ما دارند مورد استفاده قرار بگیرد همچنین داشتن اطلاعاتی مانند سال های قبل از آن مکان و یا مکان های مشابه می تواند به بهینه کردن سیستم فازی ما کمک کند.

بخش ترکیبی ماشین لرنینگ و مدلسازی فازی

❖ بخش ترکیبی ماشین لرنینگ و سیستم های فازی:

در این قسمت سعی کردیم پیش بینی هایی که با استفاده از سیستم فازی درمورد مصرف انرژی کردیم را با استفاده از تارگت کردن یکی از ستون های دیتاست مورد پرسش قرار بدهیم

```
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
y_true =scaled_data["Energy_Efficiency_%"].values

# خروجی‌های پیش‌بینی‌شده سیستم فازی
y_pred = []

for i in range(len(scaled_data)):
    temp = scaled_data["Temperature_C"].iloc[i]   # داده‌ها قبلاً اسکیل شده‌اند
    humidity = scaled_data["Humidity_%"].iloc[i]
    cooling = scaled_data["Cooling_Heating_Output_C"].iloc[i]
    hvac_eff = scaled_data["HVAC_Power_Consumption_kWh"].iloc[i]

    pred, _ = fuzzy_inference_with_weighted_average(temp, humidity, cooling, hvac_eff, fuzzy_adjustment_factor=1.0)
    y_pred.append(pred)

y_pred = np.array(y_pred)

# -------------------------
# معیارهای ارزیابی
# -------------------------
mse = mean_squared_error(y_true, y_pred)
rmse = np.sqrt(mse)
mae = mean_absolute_error(y_true, y_pred)
r2 = r2_score(y_true, y_pred)

print("MSE:", mse)
print("RMSE:", rmse)
print("MAE:", mae)
print("R²:", r2)
```

```
MSE: 0.1977946032076305
RMSE: 0.4447410518578541
MAE: 0.36561009125439603
R²: -1.366243540842849
```

توضیحات کد:

در این بخش از کد، هدف ارزیابی عملکرد سیستم فازی در پیش‌بینی کارایی انرژی است. روند کار به صورت مرحله‌ای چنین است:

1. ابتدا داده‌های واقعی هدف تعریف می‌شود. این داده همان کارایی انرژی است که با یک تبدیل ساده (کم کردن از عدد ۱) به شکل مناسب برای مقایسه آماده شده است.
2. سپس یک لیست برای خروجی‌های پیش‌بینی‌شده سیستم فازی ایجاد می‌شود.
3. در یک حلقه، برای هر ردیف از داده‌ها چهار ورودی گرفته می‌شود: دما، رطوبت، توان خروجی سرمایش/گرمایش، و مصرف برق سیستم تهویه. این مقادیر به تابع استنتاج فازی داده می‌شوند تا مقدار پیش‌بینی‌شده کارایی انرژی محاسبه شود. این خروجی‌ها یکی یکی در لیست مربوطه ذخیره می‌شوند.
4. پس از پایان حلقه، خروجی‌های پیش‌بینی‌شده به صورت یک آرایه عددی تبدیل می‌شوند تا بتوان روی آن‌ها محاسبات آماری انجام داد.

5. سپس چهار معیار ارزیابی محاسبه می‌شود:
   - میانگین مربعات خطا (MSE) برای نشان دادن میانگین توان دوم اختلاف‌ها.
   - ریشه میانگین مربعات خطا (RMSE) برای داشتن معیار خطا در همان مقیاس داده اصلی.
   - میانگین قدرمطلق خطا (MAE) برای اندازه‌گیری خطای متوسط بدون توجه به علامت.
   - ضریب تعیین (R^2) که نشان می‌دهد چه نسبتی از تغییرات داده واقعی توسط مدل توضیح داده می‌شود.
6. در نهایت این مقادیر چاپ می‌شوند تا بتوان قضاوت کرد سیستم فازی چه‌قدر عملکرد مناسبی در پیش‌بینی دارد.

روش های بهبود:

مساله اصلی آن است که در این قسمت ما داده هایی برای تارگت خودمان فرض کرده ایم و عدد های واقعی نداریم اما با این حال روش های بهبود این ها هستند:

- توابع عضویت :مرزبندی‌ها ممکن است درست انتخاب نشده باشند. استفاده از داده‌های واقعی یا الگوریتم‌های خوشه‌بندی مانند Fuzzy C-Means کمک می‌کند تا مرزها منطقی‌تر تعیین شوند.
- قوانین :قوانین ساده و گاهی متناقض هستند) وجود شرط "Any" در چند قانون .(قوانین باید دقیق‌تر تعریف شوند یا وزن‌های متفاوتی برایشان در نظر گرفته شود.
- وزن‌دهی به قوانین :در حالت فعلی همه قوانین وزن برابر دارند. اختصاص وزن‌های متفاوت بر اساس اهمیت ورودی‌ها یا داده‌های آموزش‌یافته می‌تواند خروجی را بهبود دهد.

روش دوم بدون داشتن داده های واقعی:

کد:

```python
# اعتبارسنجی مدل: رگرسیون خطی برای بهبود پیش‌بینی‌ها
def linear_regression_on_fuzzy_predictions(fuzzy_predictions, true_values):
    regressor = LinearRegression()
    regressor.fit(fuzzy_predictions.reshape(-1, 1), true_values)
    predicted_energy = regressor.predict(fuzzy_predictions.reshape(-1, 1))
    return predicted_energy

# داده‌های ورودی و پیش‌بینی‌ها
temp = 0.7  # (در بازه [0, 1]) دما
humidity = 0.4  # (در بازه [0, 1]) رطوبت
cooling = 0.5  # (در بازه [0, 1]) خروجی سرمایشی/گرمایشی
hvac_efficiency = 0.8  # (در بازه [0, 1]) کارایی HVAC
fuzzy_adjustment_factor = 1.2  # فاکتور تنظیم فازی

# پیش‌بینی انرژی
fuzzy_predictions = np.array([1.2, 2.1, 3.0, 1.5])  # پیش‌بینی‌های سیستم فازی
true_values = np.array([1.3, 2.0, 3.2, 1.4])  # مقادیر واقعی

# پیش‌بینی با مدل فازی
predicted_energy, degree_of_fuzziness = fuzzy_inference_largest_area(temp, humidity, cooling, hvac_efficiency, fuzzy_adjustment_factor)

# پیش‌بینی نهایی با استفاده از رگرسیون خطی
predicted_energy_linear = linear_regression_on_fuzzy_predictions(fuzzy_predictions, true_values)

print(f"Predicted Energy: {predicted_energy}")
print(f"Degree of Fuzziness (System Fit): {degree_of_fuzziness}")
print(f"Predicted Energy after Linear Regression: {predicted_energy_linear}")
```

```
Predicted Energy: 0.6
Degree of Fuzziness (System Fit): 0.19800000000000004
Predicted Energy after Linear Regression: [1.15952381 2.13809524 3.11666667 1.48571429]
```

توضیحات کد:

1. **توابع عضویت ورودی و خروجی**

برای دما، رطوبت، خروجی سرمایشی/گرمایشی و مصرف انرژی توابع عضویت ذوزنقه‌ای تعریف شده‌اند. این توابع مشخص می‌کنند که هر مقدار ورودی چه درجه‌ای از تعلق به دسته‌های Low، Medium یا High دارد.

2. **قوانین فازی**

مجموعه‌ای از قوانین در نظر گرفته شده‌اند که ترکیب شرایط ورودی را به خروجی انرژی (کم، متوسط یا زیاد) مرتبط می‌کنند. در این قوانین از پیش‌شرط‌های دقیق) مثل High یا (Low و پیش‌شرط‌های عمومی (Any) استفاده شده است تا پوشش بیشتری از حالات ورودی حاصل شود.

3. **استنتاج با روش Largest Area**

برای هر قانون، مقدار فعال‌سازی) با استفاده از عملگرهای min و (max محاسبه می‌شود و سپس مساحت زیر تابع عضویت متناظر در نظر گرفته می‌شود. در نهایت قانونی که بیشترین مساحت را دارد انتخاب می‌شود و بر اساس آن مقدار خروجی انرژی تعیین می‌گردد.

4. **اعمال ضریب تنظیم فازی**

خروجی انرژی انتخاب‌شده در یک ضریب تنظیم ضرب می‌شود تا مقیاس یا شدت آن متناسب با شرایط داده اصلاح شود.

5. **محاسبه درجه انطباق**

درجه انطباق سیستم بر اساس بیشترین مساحت زیر منحنی تعریف شده است. این معیار نشان می‌دهد کدام قانون بیشترین تأثیر را در خروجی داشته است.

6. **ترکیب با رگرسیون خطی**

برای بهبود پیش‌بینی‌ها، خروجی سیستم فازی با یک مدل رگرسیون خطی روی داده‌های واقعی اصلاح می‌شود. این کار باعث می‌شود خطای پیش‌بینی کاهش یابد و سیستم فازی با داده‌های واقعی هماهنگ‌تر شود.

7. **اعتبارسنجی و اجرای نمونه**

مقادیر نمونه‌ای برای ورودی‌ها) دما، رطوبت، سرمایش/گرمایش و کارای داده شده و ابتدا خروجی سیستم فازی محاسبه شده است. سپس همان خروجی‌ها با استفاده از رگرسیون خطی بهبود یافته‌اند و در نهایت نتایج چاپ شده‌اند.

این ساختار ترکیبی یک سیستم **فازی-آماری** ایجاد می‌کند که ابتدا تصمیم فازی می‌گیرد و سپس با استفاده از رگرسیون خطی نتایجش به داده‌های واقعی نزدیک‌تر می‌شود.

❖ نتیجه گیری نهایی:

در این بخش سعی شد تا با استفاده از مدلسازی فازی بتوانیم به کمک تحلیل های دیگر برویم تا اگر برای محل های تجاری، شرکت ها، خانه ها یا هر مکان دیگری که ترکیبی از این محیط ها هستند را بتوان مصرف انرژی شان را تخمین بهتری زد و بتوانیم هم با استفاده از ماشین لرنینگ و هم فازی کنترلر ها و یا ماشین های پیش بینی کننده طراحی کنیم.

در روند طی کردن هر مرحله سعی شد که منطق هر الگوریتم حفظ شود و پس از نتیجه بدست آمده سعی بر بهبود بخشی به کد کنیم این پروژه همچنان گوشه و کناری برای بسط دارد که امید است بتوان در آینده از تمامی پتانسیل های دیتاست و فازی سازی در آن استفاده کرد.